UNIVERSITY
OF SUSSEX

Masters Thesis

On the Effectiveness of Deep Learning Techniques for Malware Detection and their Resilience to Obfuscation

A thesis submitted in fulfilment of the requirements
for the degree of Master of Research in Advanced Artificial Intelligence

in the

School of Engineering and Informatics
University of Sussex

Author:
Henry Williams

Supervisor:
Jeff Mitchell

Abstract

Malicious software is an extremely pervasive threat faced by both individuals and organisations, that has the potential to cause significant damage to computer systems. We explore the usage of deep learning techniques for automated identification of malware, and evaluate how resilient these approaches are to obfuscation techniques used to bypass existing malware detection services. We present a natural language processing inspired approach, using the BERT architecture for malicious opcode sequence classification, and an image-based approach using the EfficientNet architecture. We find that these static malware detection approaches consider obfuscation as highly indicative of malware, leading to misclassification of obfuscated benign programs.

August 2025

Statement Of Originality

This report is submitted as part requirement for the degree of Masters of Research in Advanced Artificial Intelligence at the University of Sussex. It is the product of my own labour, except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged. I hereby give permission for a copy of this report to be loaned out to students in future years

Signed,

Henry Williams
Candidate No: 233561

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Malware is one of the most significant threats faced by users in the current digital landscape. The increasingly sophisticated nature of these programs, in terms of both their capabilities and their stealth, mean that this threat is only increasing as we become evermore reliant on digital infrastructure in our daily lives.

Therefore, detecting malware before it can cause damage is of high priority to security professionals. However, following the introduction of anti-virus software, malware developers have begun using various evasion techniques to avoid detection. This has led to a cat-and-mouse like game, where cyber-criminals adopt new forms obfuscation, and security researchers develop new techniques to handle them. This makes conventional approaches to malware detection challenging, due to its constantly evolving nature.

Machine learning however offers a potential means of dealing with this by finding ways to detect malware in spite of any evasion techniques. In this investigation, we evaluate various approaches across a number of machine learning domains, such as linear modelling, natural language processing, and image processing to investigate both their performance at detecting Windows malware, and their reliance on obfuscation.

We develop a BERT-based language model trained on sequences of assembly instructions, and a Convolutional Neural Network (CNN) using image representations of executables to identify malware. These are both considered static approaches as their features are extracted from the structure of the program, as opposed to behavioral characteristics observed during execution. These dynamic methods are typically more capable than static methods. However, they often incur a significant overhead in their complexity, as an environment where potentially malicious software can be safely executed is required. This typically takes the form of a remote virtual machine that isolates the programs execution, preventing it from causing harm. However, to communicate with this sandbox, an internet connection is required. In comparison, static approaches are able to operate in environments where an internet connection is not guaranteed. Therefore, in light of their disadvantages, static methods are still relevant in domain as they can be used in resouce constrained environments.

Due to the highly pervasive nature of obfuscation in modern malware, we theorise that static approaches rely heavily on the presence of obfuscation as an indicator for malware. However, obfuscation is not exclusive to malicious programs, as legitimate software developers may use it to protect their intellectual property from reverse engineering. Therefore, to better understand how static approaches identify malware, we conduct an additional experiment that aims to evaluate how our models distinguish between benign, and malicious programs.

The research questions we seek to answer in this work are as follows:

RQ1: *Are modern natural language processing techniques, such as language models and pre-training effective at identifying malware?*

RQ2: *Does pre-training improve the accuracy and reliability of malware detection models based on techniques from natural language processing?*

RQ3: *To what extent are static malware detection models resilient to obfuscation?*

1.1 Professional Considerations

This research is conducted in strict accordance with the BCS Code of Conduct (British Computing Society, 2022), ensuring adherence to the ethical, legal, and professional standards expected of computing professionals. The work is structured to protect the public interest, maintain professional competence and integrity, uphold obligations to the relevant authority, and safeguard the reputation of the profession.

Public Interest

We conduct our research with regard to the safety, rights, and welfare of the public to ensure our work is in their interest. Samples of malicious programs will be handled in a secure environment, with safeguards to protect against execution on both our own hardware, and that of others to avoid accidental infection. Furthermore, proprietary programs subject to copyright law will not be distributed to ensure the intellectual property rights of third parties are respected. Our research is conducted without discrimination on the basis of ethnicity, sex, religion, age or other protected characteristics. Finally, all materials used to conduct this investigation will be made publicly available to support transparency, reproducibility, and to benefit the wider research community.

To ensure the malware used in this investigation are unable to infect any machines used to conduct our research, we prevent them from being executed using the `chmod` program. This is a tool in UNIX based systems that modifies the permissions associated with a file, and can prevent files from being executed by the operating system.

Professional Competence and Integrity

We conduct our research within the verified level of competence of the authors, and avoid misrepresenting our skills and expertise. The project itself contributes to the continued professional development of our skills in Artificial Intelligence, and Cybersecurity. We employ established best practices in both fields where appropriate, and comply with all relevant UK legislation, including data protection, intellectual property, and cyber-security. Diverse perspectives are actively sought, and the authors welcome constructive criticism to ensure the quality and robustness of our work. No element of this research will cause harm to the property, reputation, or the livelihood of others through false or negligent actions.

Duty to the Relevant Authority

To ensure the duty of the authors to relevant authorities, we conduct this research in compliance with the policies and procedures of the University of Sussex, and any other relevant governing bodies. Any potential conflicts of interest are avoided, and disclosed in cases where they are unavoidable. We accept any professional responsibilities for our work, and that of others involved with this project. Results and findings will be reported accurately and truthfully, without distortion, omission, or misrepresentation.

Duty to the Profession

The authors acknowledge their responsibilities to uphold the credibility and reputation of the computing profession, and we pledge to take no actions that will take it into disrepute.

Chapter 2

Background

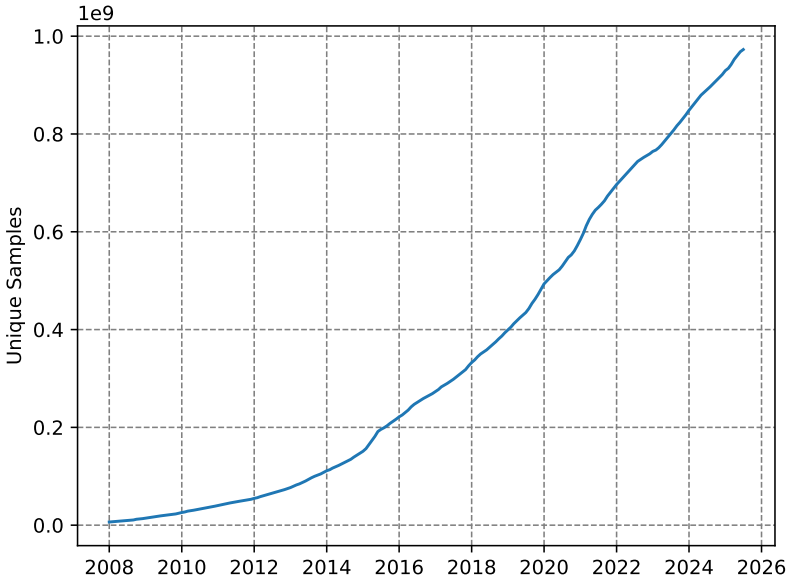


Figure 2.1: Observed growth in the number of unique instances of malware over time (AV-TEST - The Independent IT-Security Institute, 2025).

Malicious software, also known as malware, is defined as any software that has an adverse impact on the typical operation of a digital system. This could include the extraction of sensitive information, hijacking of system resources, and modification of information. They are considered one of the most pervasive threats faced by both individuals and organisations in the current digital threat landscape (European Union Agency for Cybersecurity, 2024; Alenezi et al., 2020).

This threat can been seen in infamous cyberattacks such as the 2022 Viasat hack (Quiquet, 2023), and the 2017 WannaCry incident (National Audit Office (NAO), 2017; Akbanov et al., 2019). In both of these cases, sophisticated malware were used to disrupt critical infrastructure, including communication systems across Europe, and the services used by the National Health Service.

In recent years, both the sophistication (Alenezi et al., 2020; Baezner et al., 2017) and prevalence (Figure 2.1) of malware have increased . Therefore, there has been a great deal of interest into advanced techniques for the reliable detection of malware (M. et al., 2023; Maniriho et al., 2024; Aboaoja et al., 2022) within the information security field. In this chapter, we provide a background on characteristics commonly associated with malware, and discuss various techniques proposed by security researchers to better handle this threat.

2.1 Malware

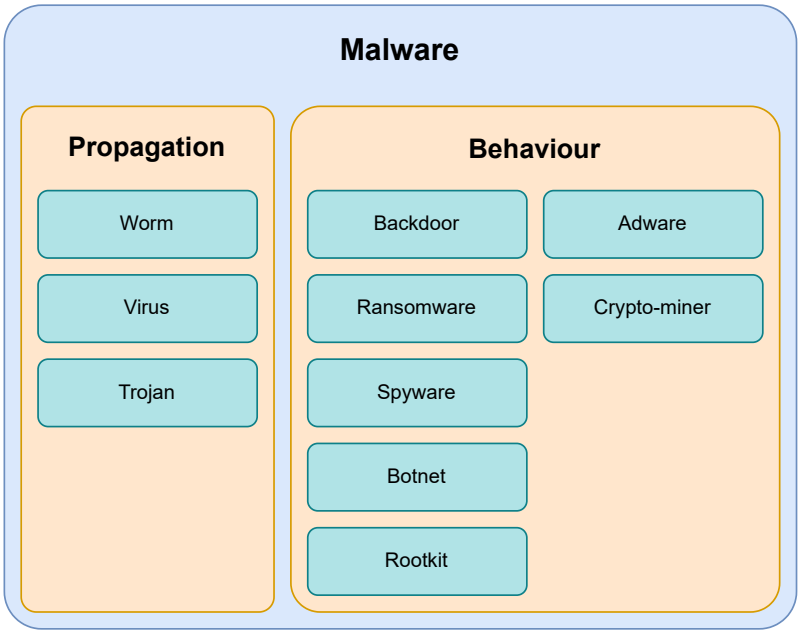


Figure 2.2: Categories of Malware, including both the means of propagation and behaviour.

As outlined in the previous section, malware is a form of software that has a detrimental impact on the expected operation of an infected system. However, both this and more formal definitions, such as the one presented in (Kramer et al., 2010), fail to define what is meant by harmful. Therefore, we define "harmful" behaviour as any actions in violation of the Confidentiality-Integrity-Availability (CIA) triad (Cochran, 2024). This is a principle that aims to guide the development of security policies to ensure the security of digital systems, and the information stored on them. It states that a system must be inaccessible to unauthorised users (Confidential), accessible to authorised users (Available), and are protected against the modification or deletion of information that it stores (Integrity). Malware provides a comprehensive means of violating this triad of security principles, and it is for this reason we use it in our definition.

Malware is categorised based on its behaviour, and how it spreads between hosts. We present the following taxonomy based on the work of (Maniriho et al., 2024) shown in fig. 2.2 that outline the different kinds of malicious behaviours, and propagation techniques. Note that categories within this taxonomy are not mutually exclusive, as malware often belong to multiple categories at once.

- **Worm** — Malicious software that automatically propagates throughout a network.
- **Virus** — Programs that modify other programs to replicate themselves throughout an infected host.
- **Trojans** — Malware that disguises itself as a legitimate program to infect a host.
- **Backdoor** — Malware that provides covert remote access to infected systems.
- **Ransomware** — Malware that denies access to systems or information until a fee is paid.
- **Spyware** — Malware that spy on the user without their knowledge, often extracting information such as credentials, keyboard logs, and live screenshots of the user's activity.

- **Botnet** — Malware that integrate infected systems into a coordinated network of agents.
- **Rootkit** — Malware providing privileged access to an infected system.
- **Adware** — Malware that display advertisements on the host machine.
- **Crypto-miner** — Malware that hijack the user's hardware to mine cryptocurrencies.

2.1.1 EVASION TECHNIQUES

The earliest forms of malware detection used a collection of unique signatures associated with malicious programs(A. Saeed et al., 2013). To counter this, the developers of malicious software began using evasion techniques that aim to deceive these systems, thereby maximising the amount of damage their software is able to do before it is identified and removed. These include obfuscation, and environmentally aware execution. In the following sections, we briefly discuss these techniques, and how they are effective in allowing malware to bypass detection by these early signature based methods.

Obfuscation

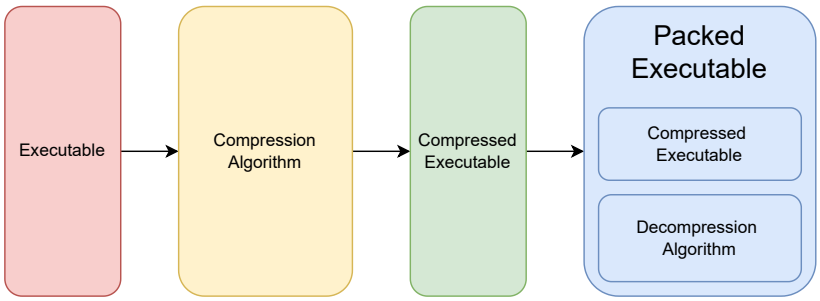


Figure 2.3: Binary packing for compression and obfuscation of executables

Obfuscation refers to the act of making something more challenging to understand to an external observer. In the context of malware, this includes techniques which alter the syntactic structure of a program to make it more difficult to understand, while maintaining its original behavioural characteristics.

Register re-assignment is one of the earliest forms of obfuscation in which the registers used by the program are randomly reallocated (Balakrishnan et al., 2005). Registers are small areas of memory within the CPU that store information used during the program's execution. By changing which registers are used to store this information, the physical structure of the program is changed without impacting its behaviour. This defeats simple malware detection solutions that use the unique cryptographic hash of a program as its signature. This is because a unique representation of a program will give a distinct hash that has not yet been seen, and therefore will not be recognised as malicious.

We can also modify the structure of a program by inserting instructions, or functions which either have no impact on the program state, or are never executed. This is known as dead-code insertion, and is a relatively common form of obfuscation, as it can provide many distinct representations of the same program. However, so-called "dead-code" can easily be identified and removed, by either human experts, or static analysis techniques such as the control flow graph (Allen, 1970).

Code transposition modifies the order of code the program (Christodorescu et al., 2004) to obscure its signatures by moving small units of code known as basic blocks. To ensure the program behaves as expected, all references to the relocated sections are updated to its new location. This

again evades detection by these early signature based methods, as many unique representations of the same program can be generated.

Despite the ability of these techniques to trick early anti-malware approaches, more sophisticated approaches are more resilient to these forms of obfuscation. These approaches use an intermediate representation, that normalise the structure of a program resulting in an identical representation for both unobfuscated, and obfuscated executables. To deceive these advanced approaches, more sophisticated forms of obfuscation are needed. One such example of this is known as binary packing, where the malicious portion of code is encoded and stored as data within another program that decoded and executes the payload (Figure 2.3). Encoding can take many forms, such as encryption or compression, but they typically make it challenging to retrieve the decoded payload without running the program. However, the portion of the packed binary responsible for decoding and execution will often remain constant, meaning it can be used for identification. Some malware developers employ the aforementioned obfuscation techniques to obscure the decoder and avoid detection which are known as Oligomorphic and Polymorphic malware (You et al., 2010).

Environmental Awareness

The techniques demonstrate the challenges of relying on the static structure of a program to determine its nature, and it is for this reason that dynamic features are considered to be more robust at identifying malicious programs. This is because the behaviour of a program is invariant to these forms of obfuscation. Dynamic features are observed at execution time and can include sequences of system API calls, modifications to the filesystem and system configuration, and internet traffic. However, these require running a potentially malicious program, which could have harmful effects on the host system. Therefore, the execution of a potentially malicious program will occur within a sandbox, an environment where harmful actions can occur with no implications on the security of the system. This sandbox is typically a virtual machine that is representative of the software's intended target.

To deceive these approaches, the malware developers will have their code check if it is running within a virtual machine before it executes. Common indicators of a virtual machine include system hardware characteristics (CPU core count, RAM size, etc), the presence of shared libraries used to communicate between the sandbox and its host, and the number of processes running on the system. If the program determines that it is likely to be running within a sandbox, it will refuse to execute the malicious portion of the code, and will exit. This prevents the observer from gaining any information about its behavioural characteristics, preventing the creation of dynamic malicious signatures.

2.2 Related Work

Malware detection is a challenging problem, and many approaches have been proposed since the first forms of malicious software were seen in the 1980s. The earliest form of malware detection used a database containing unique signatures of malicious programs. While similar signature based approaches are still in use today, there is much interest in advanced methods, that are robust to evasion techniques, and able to identify novel forms of malware (M. et al., 2023; Merabet et al., 2019; Aboaoja et al., 2022; Idika, Mathur, 2007). These advanced techniques employ a wide variety of technologies, such as multiple sequence alignment of API call sequences (Ki et al., 2015), machine learning (Kamboj et al., 2023), and deep learning.

These techniques typically fall into one of two categories, based on how the features used to classify programs are extracted, which are as follows.

- **Static** — Features extracted from the structure of a program
- **Dynamic** — Features extracted during the execution of the program

Static approaches use features extracted from the contents of an executable as opposed to its behaviour during execution. Common features used to identify malware include greyscale images of the executable (Ni et al., 2018; Jain et al., 2020; Habibi et al., 2023), printable strings (Tian et al., 2009; Singh et al., 2020; Schultz et al., 2001), and sequences of instructions (Kakisim et al., 2022; Wang et al., 2021; Darem et al., 2021).

In Schultz et al., 2001, the authors use static features such as printable ASCII strings to train various machine learning classifiers to detect malware. In their experiments, they found that the Naive Bayes algorithm was best suited to the problem, achieving 97% classification accuracy. This was one of the first papers that explore the application of machine learning in this domain, and the high accuracy of their experiment shows its potential over conventional signature and heuristic methods. However, the dataset used in this work was small, and featured an unbalanced class distribution of 3265 malicious programs, and only 1001 benign which would likely have an adverse effect on the generalisability of their proposed models. Furthermore, malware often obfuscate strings that are decoded during execution, which would be sufficient to defeat their proposed approach.

Much of the literature relating to static malware detection use convolutional neural networks (CNN) to distinguish between image-based representations of malicious and benign executables (Ni et al., 2018; Jain et al., 2020; Habibi et al., 2023). In Aslan et al., 2021, the authors represent executables as greyscale images, and train a CNN to identify images of malicious programs. While their proposed approach is able to effectively distinguish between the different types of programs, CNNs require constant input size. Therefore, resizing of the program's image representation is necessary. Malicious programs are often smaller than benign programs, meaning that in datasets where there is significant variation in size between classes, compression artifacts may be more prevalent in once class, leading to the model developing an insufficient solution. In Sharma et al., 2019, the authors also use a CNN to detect malware. Instead of using a 2-dimensional CNN however, the authors opt to use a 1-dimensional variant. This led to better performance than other CNN based approaches, as the vertical spatial dependence, which is largely irrelevant in malware detection, is removed. However, it still has the same requirement as the previously discussed approach, which could lead to a partial solution.

Instruction sequences have also been explored as static features in malware detection. For example, in (Attaluri et al., 2009; Runwal et al., 2012), the authors utilise hidden Markov models and demonstrate promising performance. As these models consider what actions are being performed by the host system without having to execute the program, they offer a solution that we believe to be more resilient to obfuscation than other static approaches.

Extending upon the idea of using instruction sequences for malware detection, (Demırcı et al., 2022) outline an approach in which they use a stacked, bidirectional Long-Short Term Memory (BiLSTM) model and pre-trained language models such as BERT (Devlin et al., 2019), and GPT-2 (Radford et al., 2019). In their investigation, the BiLSTM model was able to achieve 98% classification accuracy whereas the pre-trained transformer models achieved only 78% and 77% for BERT and GPT-2 respectively. Transformer models have been shown to be a powerful technique for sequence classification problems, and as such, this result is surprising. We believe this of the pre-training corpora used for the transformer models, which are comprised of mostly English texts. The understanding of these models would therefore be limited to languages similar to English, and may struggle to understand the intricacies of assembly language, impacting their ability to reliably classify malware. The Bi-LSTM model on the other hand, is trained from a random initialisation, and is therefore able to develop a better understanding of malicious characteristics in opcode sequences.

Previous research has also established the use of pre-trained language models in malware detection using the metadata of an executable. In (Rahali et al., 2021), the authors fine-tune BERT on text gathered from the manifest file found in android applications. This file contains information pertaining to the activities, background tasks, event listeners, and permissions granted to an application. Their model was able to achieve 97% binary classification accuracy, in comparison to (Demırcı et al.,

2022) which was only able to achieve 77% with the same model using opcode sequences. We believe this discrepancy to be a result of the language found in these manifest files being more closely related to the pre-training corpora of BERT. This would enable their model to better understand the information found in the manifest, and its use in distinguishing between benign and malicious executables. Their approach however, is limited to environments where a file containing metadata are required, such as mobile operating systems, making it unsuitable for desktop malware detection, where such metadata files are not always present.

Our exploration of the literature relating to this field demonstrate that the performance of static malware detection approaches vary depending on the features used by the model, method of classification, availability of data, among other factors. However, it is well established that they are particularly vulnerable to obfuscation. In Bacci et al., 2018, the authors investigate the impact of obfuscation on the performance of static and dynamic malware detectors on android devices. They found that while dynamic methods are not significantly impacted by the presence of obfuscation, static techniques are more affected. However, by training static methods on obfuscated programs, it is possible to mitigate this problem to some extent.

Furthermore, dynamic methods generally tend to be more capable than static methods (Damodaran et al., 2017). This is somewhat to be expected, as behaviour is more indicative of malicious actions, especially when obfuscation is used to hide a program's intent. However, dynamic methods require a sandbox to execute a program for analysis, without causing damage to a host. This typically takes the form of a remote, virtual machine as running such a sandbox locally would have a significant computational cost. As a result of this, dynamic methods require both significant compute resources, and an internet connection, which is not guaranteed. It is for this reason that static methods, that can identify potentially malicious programs, without an internet connection is needed for enhanced protection from this threat.

In addition to the high resource requirements associated with dynamic feature extraction, sophisticated evasion techniques such as environmentally aware execution provide malware developers with the capabilities to obscure the dynamic features by preventing execution within virtual machines used to extract dynamic features. Furthermore, novel obfuscation techniques, such as the one proposed by (Li et al., 2023), is able to obscure which system API calls are made by a program. This is a widely used dynamic feature for detecting malware, and the proposed technique enables the malware to call these system APIs without triggering system hooks used to track which methods are called during a program's execution. Therefore, despite offering better performance than static methods, dynamic methods are also vulnerable to evasion techniques.

2.3 Datasets

Research into malware detection depends on the availability of large scale, high quality datasets containing both malicious and benign software. However, very few of these datasets exist, and those that do often provide features derived from the programs as opposed to the executables themselves. For example, the EMBER dataset (Anderson et al., 2018) contains over a million samples of static features derived from each executable, however the binaries themselves are not distributed. Other datasets, such as BODMAS (Yang et al., 2021), despite offering fewer samples than EMBER, provide access to the executables. However, due to copyright limitations, only malicious binaries are provided.

When curating these datasets, malicious samples are easy to download en-masse through websites such as VirusShare[1] and VirusExchange[2]. These websites store large collections of malware seen in-the-wild by security professionals. However, similar large scale collections of benign software are less available, likely due to copyright protections. We have seen a number of approaches

[1] https://virusshare.com

[2] https://virus.exchange/

for curation of benign software in the literature. For example, by downloading android applications from the Google Play store (Karbab et al., 2018), scraping free software websites (Li et al., 2022), and extracting system executables. These approaches however are somewhat limited by the number of programs available on both free software websites, and the system itself. Furthermore, these executables are not guaranteed to be free of malware. Another promising approach for collecting benign Windows software is the downloading of repositories used by Windows package managers such as Chocolatey and Winget as they provide many packages that have been verified as being free of malware. However, they often distribute installers rather than the executables themselves, meaning each package has to be installed before it can be used in the dataset.

2.4 Future Research Directions

Our analysis of the literature relating to this field, has found that there are a number of promising areas of investigation. Static approaches, while less robust to structural obfuscation, are less resource intensive and are able to run on a users machine without an internet connection. Dynamic features on the other hand, offer better performance than their static counterparts, yet are still vulnerable to advanced evasion techniques that may render them less effective. Furthermore, the requirements associated with dynamic feature extraction are a significant barrier to their deployment, and demonstrate the need for efficient and effective static methods for detecting malware.

Furthermore, we identify a gap in the literature relating to the use of modern natural language processing techniques such as pre-training transformer models for malware detection on textual representations of executables. While these models have been used in this context, they use the natural language features of these programs, such as the ASCII strings, and program metadata. We will investigate the use of these techniques when trained to develop an understanding of the program itself to see if these modern NLP techniques are suited to reasoning about executables with the end goal of identifying malware.

In addition to this, we will also investigate the resilience of static approaches to obfuscation, as it seems that the it is present in most malware seen in the wild. This may cause static malware detection models to recognise harmful software based on the presence of obfuscation. This will develop a better understanding of the issues associated with static malware detection, driving further developments in the field.

Chapter 3

Methodology

In this investigation, we evaluate the effectiveness of a custom BERT-based model in static malware detection. The model is trained to distinguish between malicious and benign opcode sequences using modern natural language processing techniques such as pre-training, which are then used to identify malicious programs. We compare this model against both linear models, such as decision trees, and other static machine learning methods seen in the literature, such as Convolutional Neural Networks. Finally, we evaluate to what extent the opcode sequence classification, and the CNN image classification models rely on the presence of obfuscation as an indicator of malicious software.

3.1 Dataset

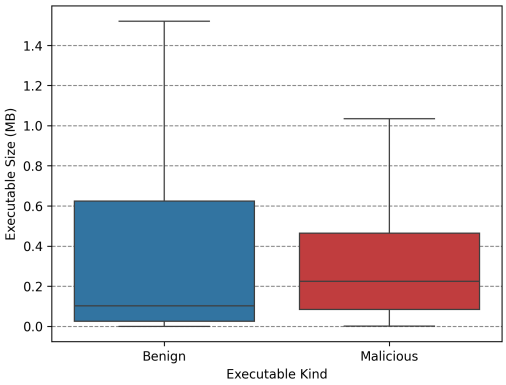
Our dataset consists of 11115 Windows PE32 files (See appendix A.6), with a class distribution shown in fig. 3.1c. Malicious files are gathered from VirusShare[1], which contains samples from real-world cybersecurity incidents, providing approximately 90 million samples of which nearly 6000 were randomly selected. The benign executables were collected from the personal devices of the authors, and were scanned using Microsoft defender to ensure no mislabelled samples were present. This includes various software packages, games, developer tools, system programs, and other miscellaneous executables to ensure the dataset is representative of the kinds of benign programs commonly seen on the machines of potential end users.

We opt to investigate Windows malware detection due to the operating systems widespread usage, and therefore high availability of malware targeting it. Malware targeting other operating systems, such as Linux of MacOS do exist, but there are significantly fewer samples, meaning the availability of data is reduced.

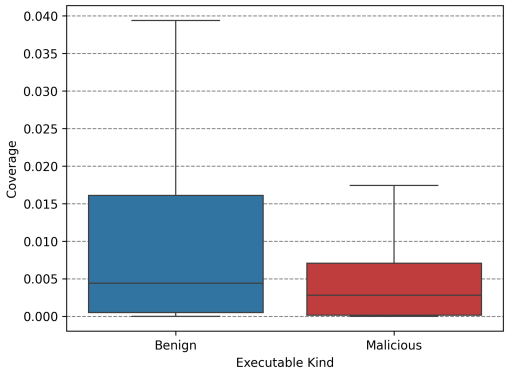
The dataset demonstrates wide variation in characteristics between classes. For example, we find that malicious programs are often much smaller than benign executables, as shown in fig. 3.1a. We believe this to be because benign programs typically provide more functionality, such as graphical user interfaces, telemetry, and compatibility, which increase the size of the program. Malicious software on the other hand, are developed with a single goal in mind, reducing its size.

We hypothesize that both our dataset, and other datasets containing malicious software will demonstrate variation in the prevalence of obfuscation between malicious and benign executable classes. To investigate this, we perform static analysis of the samples in our dataset using Rizin (Rizin Organization, 2025) to find the code analysis coverage. This metric gives the ratio of code analysed by the tool, to the total amount of code in an executable. In obfuscated programs, this result is typically much lower than non-obfuscated executables as it is unable to perform its analysis due to the non-standard structure. The results of this analysis are presented in fig. 3.1b, which supports our hypothesis, and indicates malicious samples are more frequently obfuscated than benign samples.

[1] `https://virusshare.com/about`



(a) Distribution of executable size across classes.



(b) Distribution of code analysis coverage across classes.

(c) Distribution of classes with our dataset.

Class	Count
Malicious	5792
Benign	5323
Total	11115

Figure 3.1: Description of our dataset, including the size of programs (a), the extent of obfuscation (b), and the total number of samples in each class (c).

The dataset is then split to produce a training set, and evaluation set at a ratio of 80% to 20% of samples respectively. To ensure consistent class distribution between these sets, samples are shuffled prior to splitting.

3.1.1 OBFUSCATION EXPERIMENT DATASET

Table 3.1: Distribution of classes with our secondary dataset.

Class	Count
Malicious	60
Benign	61
Total	121

To investigate the impact of obfuscation on malware detection, we also curate a secondary dataset, consisting of unobfuscated executables. We use the work presented in (Rokon et al., 2020) to identify GitHub repositories containing malicious source code. The authors provide a list of URLs to these repositories, from which we select Windows programs that provide an executable and with no mention of obfuscation in its description or codebase. Benign samples consist of windows system binaries not present in our original dataset. We identify a collection of approximately 60 Windows XP system binaries which are free of obfuscation for this secondary dataset. This gives a selection of 121 programs, each of which are free of obfuscation for use in our later experiment into the resilience of machine learning algorithms to obfuscation.

3.2 Metrics

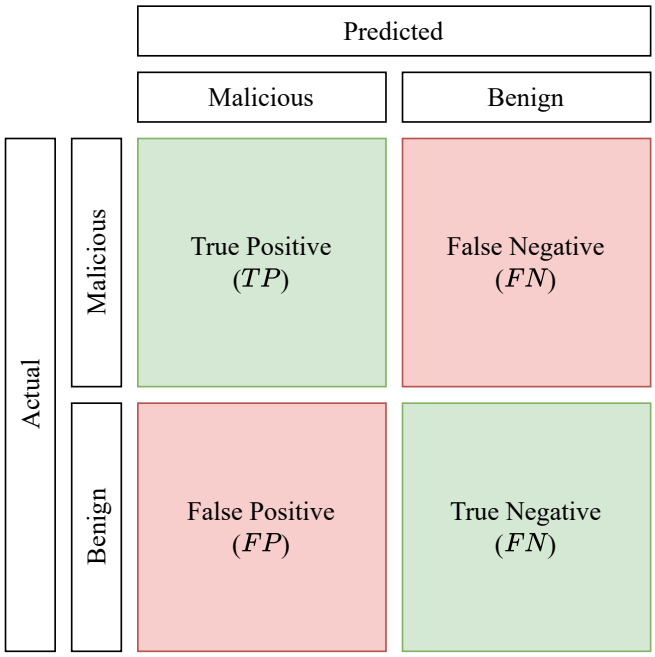


Figure 3.2: Confusion Matrix describing the kinds of classification results in malware detection.

In our investigation, malware detection is presented as a binary classification problem, where our models aim to distinguish between malicious and benign software. To quantify the performance of the various approaches tested in this investigation, we find the accuracy, F1-score, precision, and recall eq. (3.1). These metrics are based on the different kinds of results a binary classifier can produce, shown in fig. 3.2.

Accuracy is the ratio of correct classifications to the number of samples tested, where a score of "1" would be a perfect classifier for a given set of samples. However, the value of this metric can be misleading in unbalanced datasets.

Precision and recall both operate on the class-level, therefore to reduce these metrics to a single number, we typically take the average of these values for each class in the dataset, known as macro-averaging. Precision is defined as the ratio between the number of correctly identified samples, to the total number of samples predicted as belonging to a given class. Recall on the other hand is the number of correct predictions, to the number of samples in given class. F1 score, acts as an aggregation of these values, and is given by the harmonic mean.

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ \text{Precision} &= \frac{TP}{TP + FP} \\ \text{Recall} &= \frac{TP}{TP + FN} \\ \text{F1-Score} &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned} \tag{3.1}$$

3.3 Baseline

Similarity	Malicious	Benign
Malicious	0.1428	0.0931
Benign	0.0931	0.1217

(a) Cross-class external function similarity.

Similarity	Malicious	Benign
Malicious	0.2249	0.1606
Benign	0.1606	0.1643

(b) Cross-class DLL similarity.

Figure 3.3: Cosine similarity between classes based on external functions and DLLs.

We firstly establish a baseline using conventional machine learning algorithms on the lookup table associated with each program. This lookup table tells the program where external references, such as libraries and functions exist in memory. These external references offer functionality that would otherwise be inaccessible to the program. For example, to interact with the file-system, the lookup table will include a reference to `KERNEL32.dll`, which provides the address of the functions needed to create, read, and write to the file-system. We can extract all functions and libraries used by an executable with other static analysis tools such as Radare2 (Radare Organization, 2025). References to functions that occur once across the dataset are removed as approximately half of all functions are seen exactly once across the entire dataset. This gives two binary feature vectors for each sample in the dataset, telling the classifiers which external functions, and libraries are used by the program. We test 3 machine learning algorithms to establish this baseline, which are support vector classifiers, logistic regression, and decision trees.

We also use the feature vectors to quantify the similarity of programs in our dataset with the pairwise cosine similarity function (eq. (3.2)). We then find the average similarity between classes which are presented in fig. 3.3. From this, we can assume that functions and libraries used by the malicious programs are more similar than the ones used by benign programs. Furthermore, functions used by malicious programs are vastly different to functions used by benign programs.

$$S(\mathbf{A}, \mathbf{B}) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} \tag{3.2}$$

3.4 RoBERTa for Opcode Sequence Classification

```
ret lea mov add add lea mov add add nop
```

Figure 3.4: An example sequence of opcodes

In recent years, transformer-based architectures for natural language processing have been shown to be highly effective across a wide variety of language based tasks. A significant advancement within the field was the development of the Bidirectional Encoded Representation from Transformers model, known as BERT (Devlin et al., 2019). This model uses an encoder-only architecture, based on the multi-head self-attention mechanism (Vaswani et al., 2023) in conjunction with a pre-training stage to develop a comprehensive understanding of language. Pre-training consists of two tasks, masked language modelling where the model predicts hidden tokens based on their surrounding context, and next sentence prediction. The pre-trained model can then be fine-tuned on a downstream problem such as sequence classification, question answering, and general language understanding. At the time of publication, it achieved state-of-the-art performance in the General Language Understanding Evaluation (GLUE), Stanford Question Answering Dataset (SQuAD), and Situations With Adversarial Generations (SWAG) benchmarks and has been used extensively throughout the natural

language processing field. Fine-tuning of the pre-trained model allows researchers to solve complex tasks with fewer training samples, short training periods, and less intensive compute requirements. RoBERTa extends upon the original BERT model by expanding the scale of the model, removing the next sentence prediction task from its pre-training stage, optimising the hyperparameters, and training on a larger corpus.

We develop a custom RoBERTa model, trained on opcode sequences for malware detection. Opcodes refer to the operation to be performed by the CPU, such as `add`, `cmp`, `mov`, written in a human readable form. To generate these sequences, we use Radare2 to extract the disassembly of a program (See algorithm 1), and remove the operands of each instruction (algorithm 2) to reduce the vocabulary size and therefore the computational resources needed to train the model. This gives a sequence of opcodes representing each program within our dataset, as shown in fig. 3.4. The sequences are then tokenized at the word level where each opcode is assigned a single token.

We train four of these models that each take a tokenized opcode sequence of a fixed length. The first model is trained in masked language modelling, which is then used to train a sequence classification model, referred to as the "pretrained" model throughout this report. We also train another pre-trained sequence classification model which has had the weights in its attention layers frozen, referred to as the "frozen" model to determine if the representations produced by the attention layers are sufficient for distinguishing between malicious and benign opcode sequences. Finally, we also train a RoBERTa sequence classification model from a random initialisation, to assess if the pre-training step is necessary, referred to as the "base" model.

Our models share the architecture shown in table 3.2 that was found to be the optimal configuration after an extensive hyperparameter sweep of 90 runs. Training was performed with a single Nvidia A40 GPU, and 16GB of ram with a batch size of 128 for a single epoch, using the AdamW optimizer, and Cross-Entropy loss function provided by Pytorch.

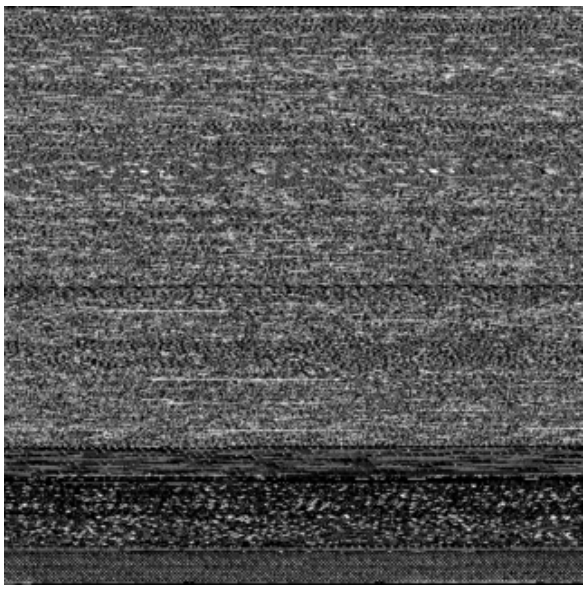
For each opcode sequence, the model predicts two values, the likelihood of the sequence being malicious, or benign where the larger of these two values is its predicted class. However, the full opcode sequence of a program is too long to be used as the models input. Therefore, we split the full sequence of opcodes into multiple sub-sequences of a fixed length, and aggregate the logits to classify a program. We test both raw logit mean aggregation, softmax logit mean aggregation, and majority voting, and found raw logit mean aggregation to be most effective.

Table 3.2: RoBERTa sequence classification model parameters

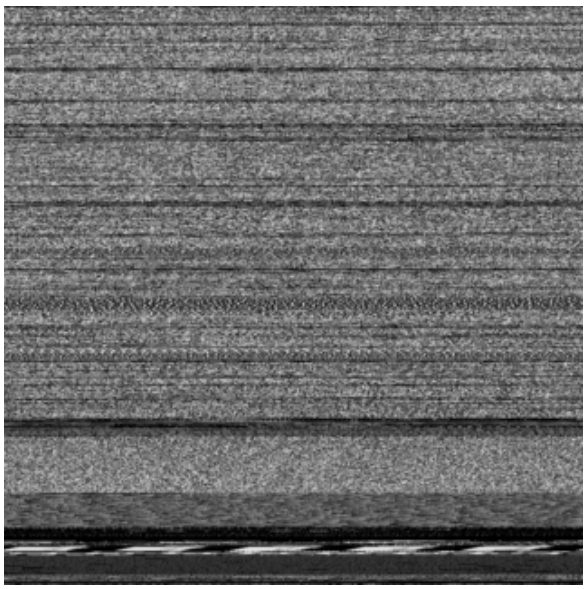
Parameter	Value
Sequence Length	32
Hidden Layer Size	512
Hidden Layers	7
Attention Heads	4
Intermediate Layer Size	2048
Activation Function	GELU
Hidden Dropout Rate	0.01
Attention Dropout Rate	0.001

3.5 CNN for Image Classification

In addition to the RoBERTa sequence classification model, we also evaluate an image based approach. We use the EfficientNet-B0 CNN architecture (Tan et al., 2020), due to its computational efficiency



(a) Image representation of a benign executable.



(b) Image representation of a malicious executable.

Figure 3.5: How images can be used to represent programs.

and high performance on image classification problems. The model is trained from a random initialisation, and the final layer of the model is modified to output a binary class likelihood values. Programs are converted to greyscale images using the procedure outlined in appendix A.3, converted to RGB, and resized to 320×320 pixels before it is used by the model, as shown in fig. 3.5.

As previously established, the size of benign programs is typically much higher than malicious programs. Therefore, benign images are resized to a greater extent to that of malicious images. Resizing an image will introduce artifacts to the image, such as anti-aliasing or blurring. Thus, benign images may have more prevalent artifacts than malicious images which could be used by the model to learn a suboptimal solution. To prevent this from occurring, we train the same model on a subset of the dataset where the distribution of program size between classes are more similar, known as the "filtered" model. We use a genetic algorithm that minimises the Kullback-Leibler divergence D_{KL} between the distributions of program size for each class of program by modelling it as a Knapsack problem (Khuri et al., 1994). We also encourage the algorithm to select a subset with an approximately equal number of malicious and benign samples to ensure class balance.

We train both the base model, and the filtered model trained for 20 epochs, with the binary cross entropy loss function and a batch size of 32. Training was conducted using an Apple M2 Pro CPU, leveraging the metal performance shaders backend for hardware acceleration, with 16GB of RAM using the AdamW optimiser.

3.6 Obfuscation

In modern malware, obfuscation is highly pervasive. As a result of this, we believe that static malware detection methods learn to identify obfuscation as opposed to other malicious characteristics. However, obfuscation is not exclusive to malicious programs, as legitimate software developers use similar techniques to protect their intellectual property. Should this be the case, legitimate benign software that have been obfuscated may be misidentified as malicious, thereby degrading the effectiveness of these approaches in real world deployment.

We investigate this by observing how the models performance changes as we increase the presence of obfuscation within the unobfuscated dataset. We perform three trials for each model, that vary which kinds of program are to be obfuscated. For example, only malicious, benign, and all programs. At each step, we obfuscate a certain percentage of programs within the dataset and this percentage increases by 10% upon completion. We collect the logits for each sequence, which are used to calculate both the sequence level, and the file level metrics. We also perform this experiment on the image classification model to see if these models are better suited to obfuscation resilient malware detection.

Obfuscation is performed using a tool known as UPX (Markus F.X.J. Oberhumer et al., 2025), which is a binary packing tool. We chose this tool due to its extensive compatibility with Windows executables. Furthermore, a number of our malicious executables have been identified as using this tool for obfuscation, and it has also been used to compress and obfuscate benign files. Therefore, we feel that it is a suitable tool to replicate the forms of obfuscation used by both malicious and legitimate software developers seen in the wild without having to implement our own obfuscation tooling.

Chapter 4

Results

4.1 Baseline

Table 4.1: Performance of baseline methods for malware detection using the import tables of executables

Algorithm	Feature	Accuracy	F1 Score	Precision	Recall
Support Vector Classifier	DLL	0.8859	0.8856	0.8856	0.8856
	Function	0.9248	0.9248	0.9266	0.9262
Logistic Regression	DLL	0.8682	0.8682	0.8687	0.8695
	Function	0.9438	0.9438	0.9442	0.9447
Decision Tree	DLL	0.8877	0.8876	0.8876	0.8885
	Function	0.9406	0.9406	0.9415	0.9417

The results of our baseline approaches are presented in table 4.1. These results indicate that across each of the machine learning algorithms tested, the external function features clearly offer better performance over the library level features. We find that the logistic regression, and decision tree algorithms, using the external function feature vector are comparable in terms of accuracy, and the support vector classifier offers worse performance by a margin of 0.02 in its F1 score.

When the library level information are considered however, we find that both the support vector machine and the decision tree algorithms provide similar performance of approximately 0.88 F1 score. The logistic regression algorithm however, is marginally worse at identifying malware by 2%.

However, these models are particularly vulnerable to obfuscation as they rely of features extracted from the import table of an executable. Binary packing fully obfuscates this information, as it is found within the encoded portion of the packed executable, therefore unable to be extracted using static methods. Therefore, should we try to evaluate these models on a set of packed executables, the feature vectors used to represent executables would be empty, making it impossible to identify malicious programs.

4.2 Sequence Classification

In table 4.2 we present both the sequence level, and file level classification metrics of our machine-code language models. We find that in sequence level tasks, where the model is tasked with clas-

Table 4.2: Performance of our RoBERTa model for opcode sequence classification and malware detection

Model	Sequence Level				File Level			
	Accuracy	F1-Score	Precision	Recall	Accuracy	F1-Score	Precision	Recall
From Scratch	0.8756	0.8742	0.8798	0.8756	0.8976	0.8971	0.9095	0.8989
From pretrained	0.9010	0.9002	0.9042	0.9010	0.9452	0.9452	0.9479	0.9458
From pretrained (w/ Frozen Attention Weights)	0.8302	0.8265	0.8409	0.8302	0.8076	0.8037	0.8393	0.8098

sifying a subsequence of opcodes as either malicious or benign, the pre-trained model offers the best performance. When this pre-training stage is skipped, performance degrades by approximately 3%, indicating the prior understanding of opcode sequences is necessary to more accurately identify malicious sequences. In comparison to our baseline approaches presented in table 4.1, only the pre-trained model offers similar performance.

In addition to this, the performance of the same pre-trained model is impacted significantly when the parameters of its attention layers are frozen. These weights encode the understanding of language developed during its pre-training stage to create an encoded representation of a sequence, which is then used by a classification head to determine its classification. In certain fine-tuning tasks, these initial layer weights are frozen to reduce the computational cost, and the amount of data used to train the model. However, we see that freezing the attention layers has a significantly negative impact on the models performance.

These sequence level tasks however are not representative of our models performance in real world tasks. Therefore, to understand how they perform in real-world malware detection, we are required to reduce multiple sequence classifications across an entire program, into a single file-level classification. By taking the mean of each, un-normalised pair of class logits, we are able to find this file-level classification. From this, we find that the pre-trained model achieves the best file-level performance, with an F1 score of 0.94, outperforming our baseline approaches by a small margin. This increase in performance between sequence level and file level tasks is also seen in the non-pretrained model to a lesser extent. However, the model with frozen attention layers exhibits a decrease between sequence level, and file level performance.

4.3 Image Classification

Table 4.3: Performance of the image classification models for malware detection

Dataset	Accuracy	F1-Score	Precision	Recall
Full	0.9316	0.9315	0.9314	0.9322
Filtered	0.9306	0.9306	0.9309	0.9306

Our results demonstrate that both image classification models do not outperform our baseline, as shown in table 4.3. Furthermore, despite outperforming the scratch and frozen RoBERTa opcode sequence classification models in file level tasks, the pre-trained model achieves a better F1 score by approximately 0.014. This suggests that both conventional machine learning algorithms, and custom pre-trained language models offer a more powerful solution to this problem.

The same model was also trained on a subset of the original dataset, found by the genetic algorithm outlined in appendix A.4. We theorise that, due to the discrepancy in the size of programs in both classes, the model would use rescaling artifacts as an indicator of an executables class. The

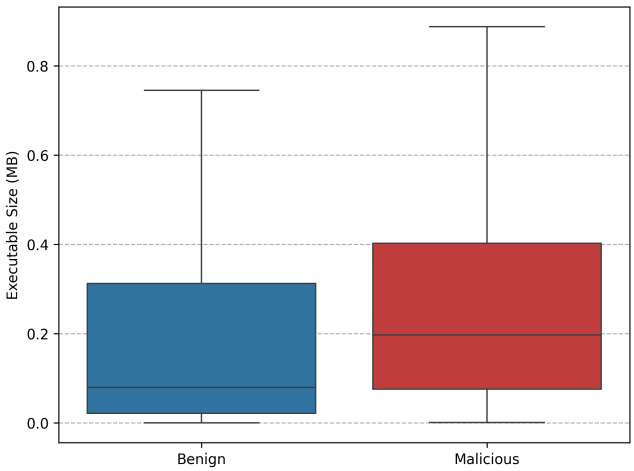


Figure 4.1: Distribution of executable size in the filtered dataset

subset of our original dataset has an approximately equal distribution in program size between malicious and benign executables, as shown in fig. 4.1, and maintains an acceptable class balance between the two. However, we find that the model trained on this filtered dataset, has very little impact on its performance when evaluated on the original dataset. This indicates that the model is not learning to identify malicious and benign programs based on any artifacts that originate from rescaling.

4.4 Obfuscation Experiment

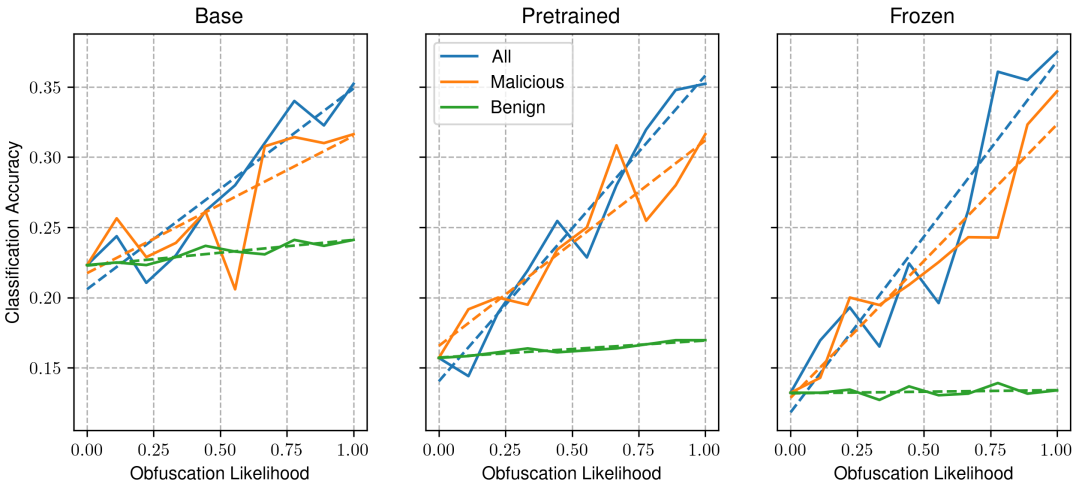
To better understand how these approaches learn to distinguish between malicious and benign programs, we conduct an investigation into how their performances changes as obfuscation becomes more prevalent in a dataset of unobfuscated programs. Both the RoBERTa model, and the Image Classification models are tested, however we are unable to test the performance of our baseline approaches, as the tool used to obfuscate programs prevents feature extraction.

4.4.1 Sequence Classification

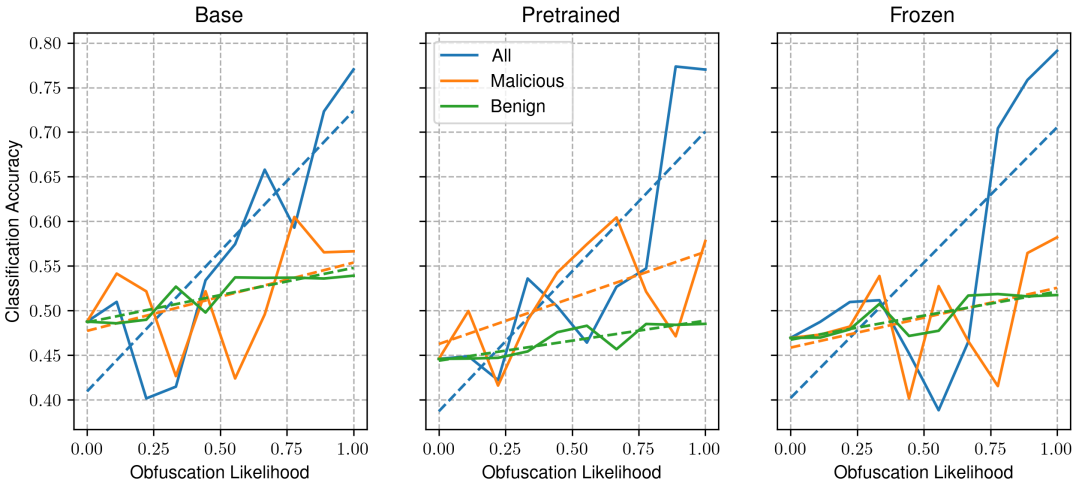
We find that our sequence classification models seem to heavily rely on obfuscation as an indicator of a programs intent. This can be seen in fig. 4.2a, where an increased presence of obfuscated malicious samples corresponds with an increase in classification accuracy. When only benign files are obfuscated however, we see no meaningful trend.

However, when sequence level accuracy is considered, this trend is pronounced to a lesser extent, as shown in fig. 4.2b. We do see an a positive correlation between classification accuracy and obfuscation likelihood across all samples. However when obfuscation is applied to a single class of programs in the dataset, no correlation is seen.

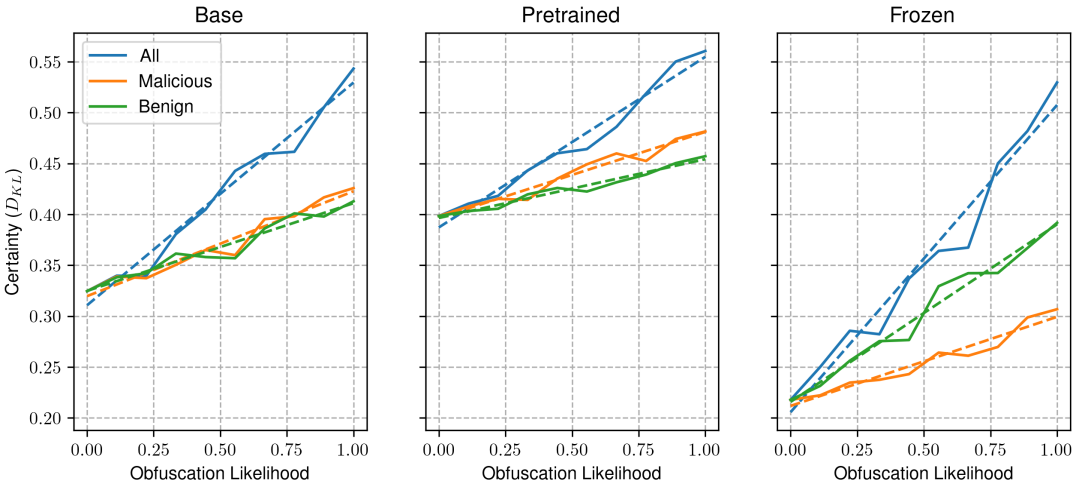
To better understand how our models handle obfuscation, we quantify the certainty in their predictions by taking the Kullback-Leibler Divergence (eq. (4.1)) between the models output, and the uniform distribution. This metric tells us how confident a model is in its predictions, by measuring the distributional difference between its predicted class likelihood distribution, and a uniform distribution representing the model picking at random. We find that certainty increases significantly across all trials as we increase the prevalence of obfuscation as shown in fig. 4.2c. This suggests that the model considers obfuscation as highly indicative of a samples class.



(a) File-level performance



(b) Sequence-level performance

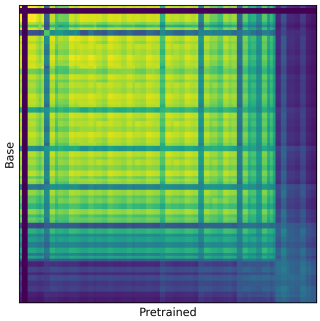


(c) Confidence in model predictions

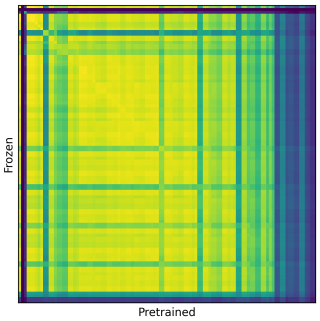
Figure 4.2: Impact of obfuscation on the performance of file and sequence level classification, and certainty of the RoBERTa opcode sequence classification models.

$$D_{KL}(p \parallel q) = \sum_{x \in \mathcal{X}} p(x) \log\left(\frac{p(x)}{q(x)}\right) \tag{4.1}$$

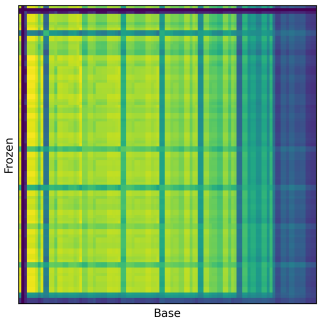
We also see that across all models tested in this experiment, the trends are largely similar. For example, our results show that as the presence of obfuscated malicious samples increase, the classification increases accordingly. This trend is seen in all the models used in this experiment, however the extent of the trend varies between them. This suggests that our models are learning similar representations, yet are limited by other factors such as a reduced prior understanding of the language, and insufficient model complexity. To visualise this, we perform representational similarity analysis (RSA) on the activation's of our models at each layer using Centered Kernel Alignment (CKA, shown in appendix A.5) as outlined in (Kornblith et al., 2019). This metric quantifies the correspondence between hidden representations of data across models trained from different initialisations. This is shown in fig. 4.3, and indicates that the similarity of representations in the attention layers of the pre-trained and non-pretrained models decrease with the number of layers, and that the representations at the classification heads are very similar. Our frozen model however, demonstrates a higher extent of representational similarity in the attention layers, and significant dissimilarity at the classification head.



(a) RSA of pre-trained and non-pretrained models.



(b) RSA of pre-trained and frozen models.



(c) RSA of frozen, and non pre-trained models.

Figure 4.3: Representational Similarity Analysis (RSA) of our baseline models using Centered Kernel Alignment.

We then repeat this experiment on two identical groups of models trained on modified datasets to see if data augmentation can produce models which are more resilient to obfuscation. The first of these datasets has had all of its benign samples obfuscated prior to opcode extraction, and the second

contains both the obfuscated and regular benign samples. We refer to these as our obfuscated benign and partially obfuscated datasets respectively.

Obfuscated Benign

The models trained on this dataset all demonstrate increased classification accuracy when only benign samples are obfuscated. When only malicious programs are obfuscated however, its accuracy decreases. In addition to this, when all samples are targets for obfuscation, the file level accuracy remains relatively consistent at approximately 50%, suggesting random choice as shown in fig. 4.4a. Furthermore, the sequence classification metrics demonstrate that accuracy also remains consistent as we increase the number of obfuscated benign samples, whereas it decreases as more malicious samples are obfuscated. The certainty of the models in these predictions indicate that increasing the presence of obfuscation leads to reduced confidence in its predictions.

Partially Obfuscated Benign

Models trained on the partially obfuscated benign dataset demonstrate more consistent file-level classification accuracy when only malicious samples are obfuscated as opposed to the fully obfuscated benign models, which decrease under the same conditions (Figure 4.5). However, this is not the case for the frozen model, as we see a significant decrease in accuracy with increasing presence of obfuscated malicious samples.

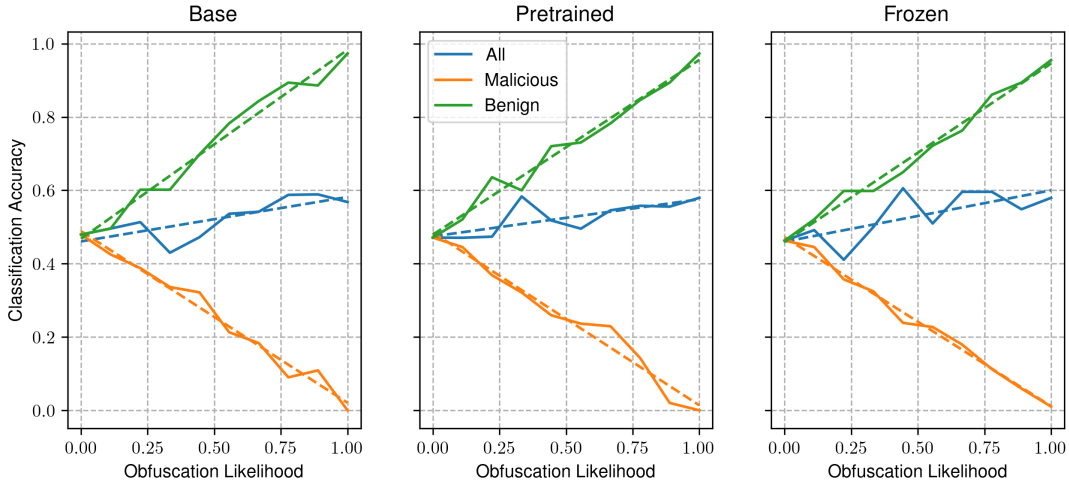
Furthermore, as was the case in the obfuscated benign models, accuracy increases with the number of obfuscated benign samples in both the pre-trained, and frozen models. The model trained from a random initialisation however, exhibits a unique trend in its accuracy across different obfuscation targets where it increases consistently across all targets. In comparison to the other models tested in this experiment, where we see highly similar trends between different models trained on the same data, that differ only in their extent. However, in this case, we see a unique trend in the model trained from scratch, where classification accuracy marginally increases consistently across each obfuscation target. The pre-trained, and frozen pre-trained models however, show a significant increase in accuracy when benign samples are more commonly obfuscated.

However, sequence level accuracy tells a different story. Between all models, we see sequence level accuracy decrease as the number of obfuscated malicious samples increase. We also see that the observed increase in accuracy with the number of obfuscated benign samples is less pronounced than the file level metrics. Futhermore, as we increase the number of obfuscated samples in both classes, there is no discernible trend in the models accuracy.

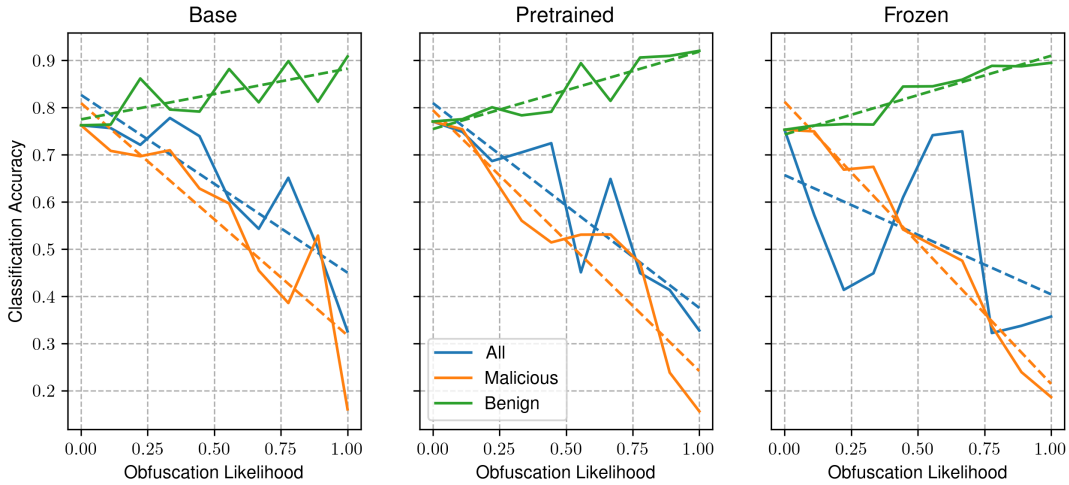
We also see the model confidence, decreasing with obfuscation likelihood across all targets. This can also be seen in the obfuscated benign models.

4.4.2 IMAGE CLASSIFIER

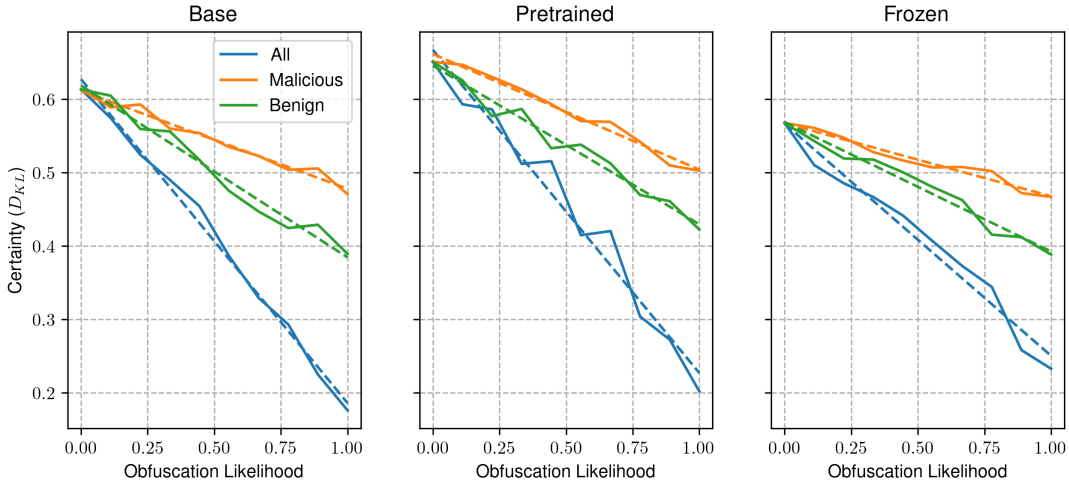
The impact of obfuscation on our image classification models is shown in fig. 4.6. We find that both of the image based malware detection models are more likely to identify malicious software based on the presence of obfuscation. In both cases, we see that the presence of obfuscation increases across our malicious samples is positively correlated with the classification accuracy. In comparison to this, an increased prevalence of obfuscation in benign samples leads to a degradation in performance. Furthermore, when obfuscation is applied across all samples in this dataset, we see no meaningful trend in classification accuracy as obfuscation becomes more common.



(a) File-level performance

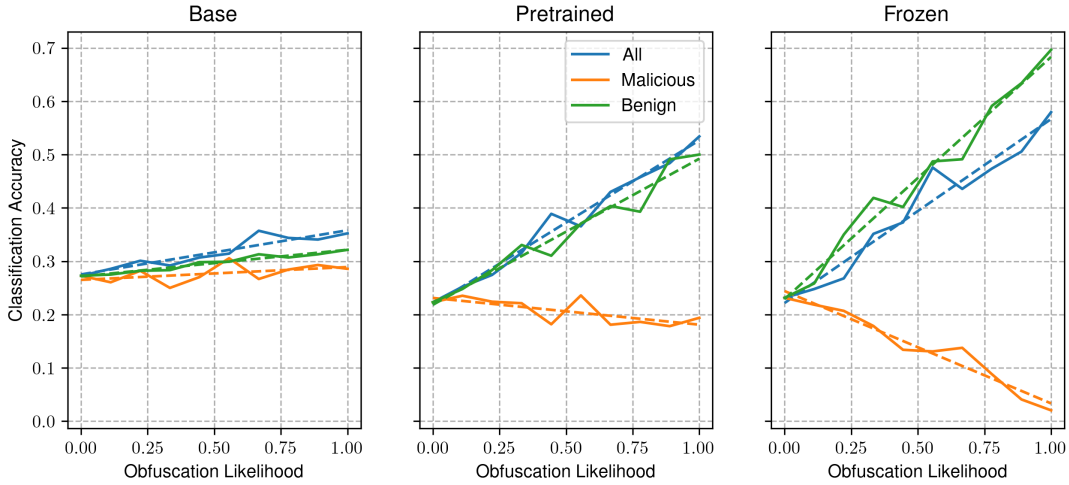


(b) Sequence-level performance

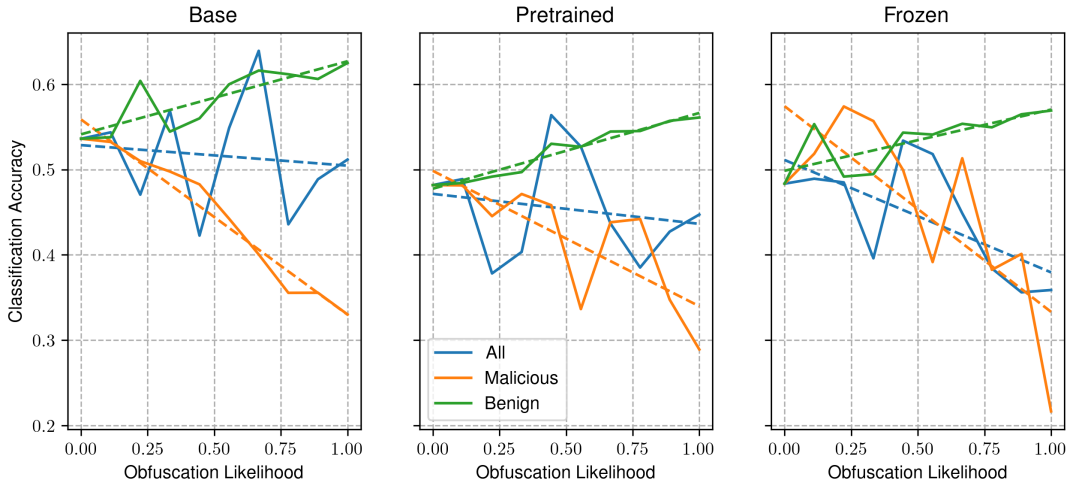


(c) Confidence in model predictions

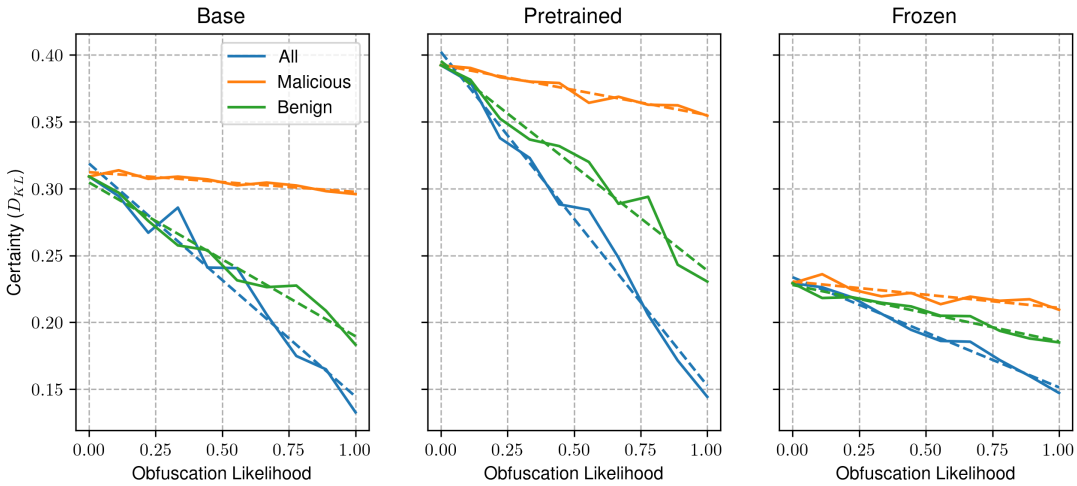
Figure 4.4: Impact of obfuscation on the performance of file and sequence level classification, and certainty of the RoBERTa opcode sequence classification models trained on the obfuscated benign dataset.



(a) File-level performance

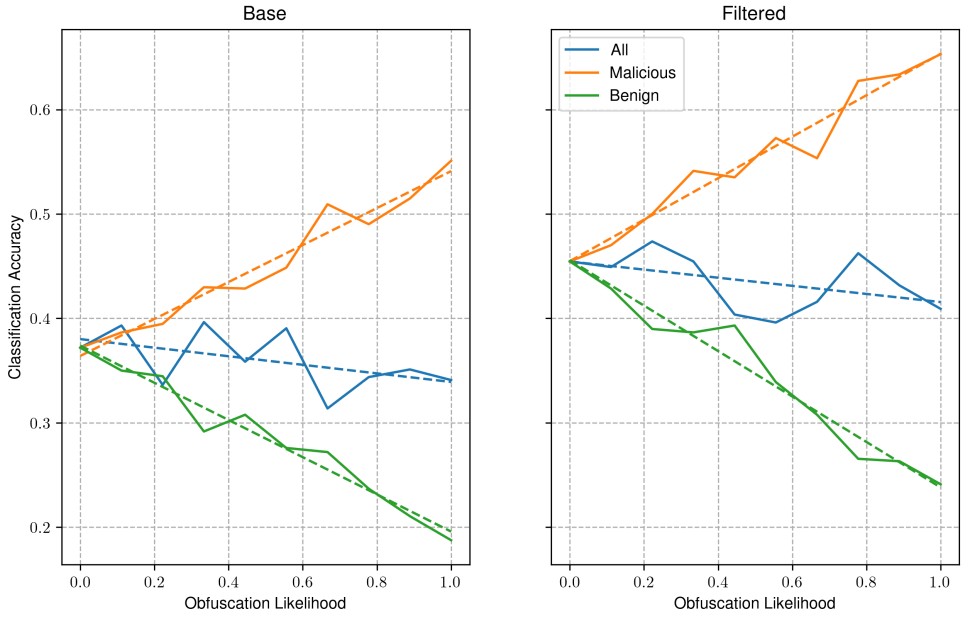


(b) Sequence-level performance

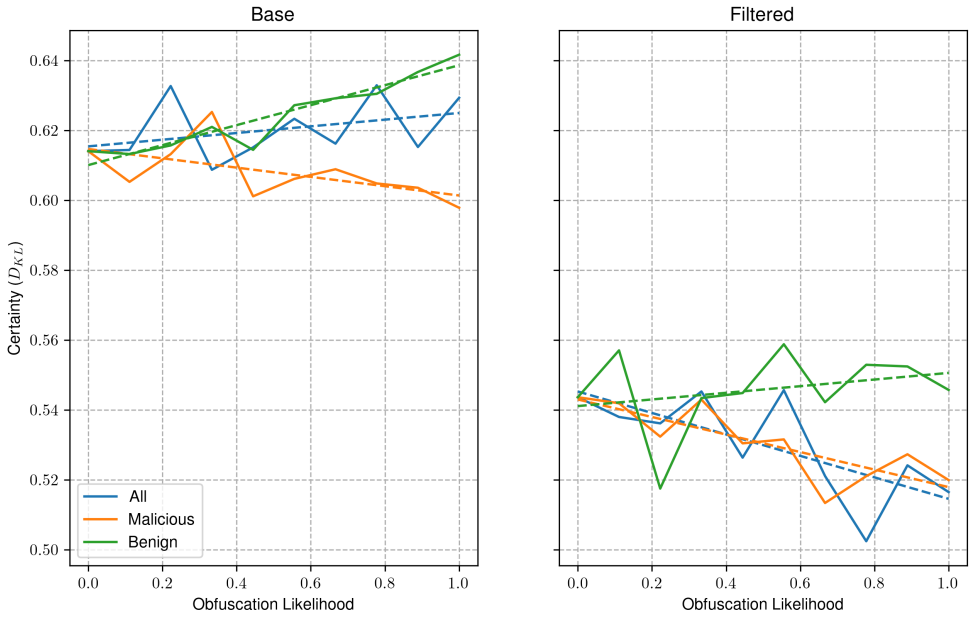


(c) Confidence in model predictions

Figure 4.5: Impact of obfuscation on the performance of file and sequence level classification, and certainty of the RoBERTa opcode sequence classification models trained on the partially obfuscated benign dataset.



(a) Classification accuracy



(b) Confidence in model predictions

Figure 4.6: Impact of obfuscation on the performance of image-based malware detection models, and the confidence of the model in its predictions.

Chapter 5

Discussion

In this research, we assess the suitability of deep learning techniques for static malware detection, and the reliance of these approaches on obfuscation. We evaluate three solutions to this problem, consisting of linear machine learning models, transformer based sequence classification models, and an image classification model using a CNN. Each of these approaches differ in the amount information provided to the model. For example, our baseline models use very little information when compared to the information used by CNN and Transformer based approaches.

Linear models comprise our baseline against which all other approaches are compared. We test a logistic regression, decision trees, and support vector machine with a radial basis function (RBF) kernel. Each of these models are trained on information extracted from the import tables of programs in our dataset, informing the models as to which external functions or libraries are used by an executable.

The sequence classification models use modern natural language processing techniques, such as transformer based language models and pre-training. We use a RoBERTa, an optimised variation of the widespread BERT architecture, trained on sequences of assembly opcodes. We assess the impact of pre-training by training two sequence classification models, one using a pre-trained language model, and another trained from a random initialisation. We also train the pre-trained model which has had all parameters in its attention layers frozen to assess if the representations of the pre-trained model are sufficient at identifying malicious opcode sequences, reducing the computational cost of training.

Convolutional Neural Networks, specifically the EfficientNet architecture, offer a more computationally efficient approach to this problem. This model is trained on image representations of the executables in our dataset. We test this model, and a model trained on a subset of the dataset with a more similar distribution in the size of programs between classes. This was conducted to determine if the model uses rescaling artifacts to distinguish between malicious and benign programs, as the latter are typically larger than malware.

We then test these models on a secondary dataset of un-obfuscated programs, and increase the presence of obfuscation in each class to see if the presence of obfuscation indicative of malware. In this chapter, we provide our interpretation of the results, its relation to the work of others, and the limitations of this investigation.

5.1 Interpretation of Results

Baseline

We find that for each of the approaches which are used to establish our baseline, function level features outperform library level features, by a significant margin. On average, the difference between the same model trained on external function usage, and external library usage, the accuracy of the

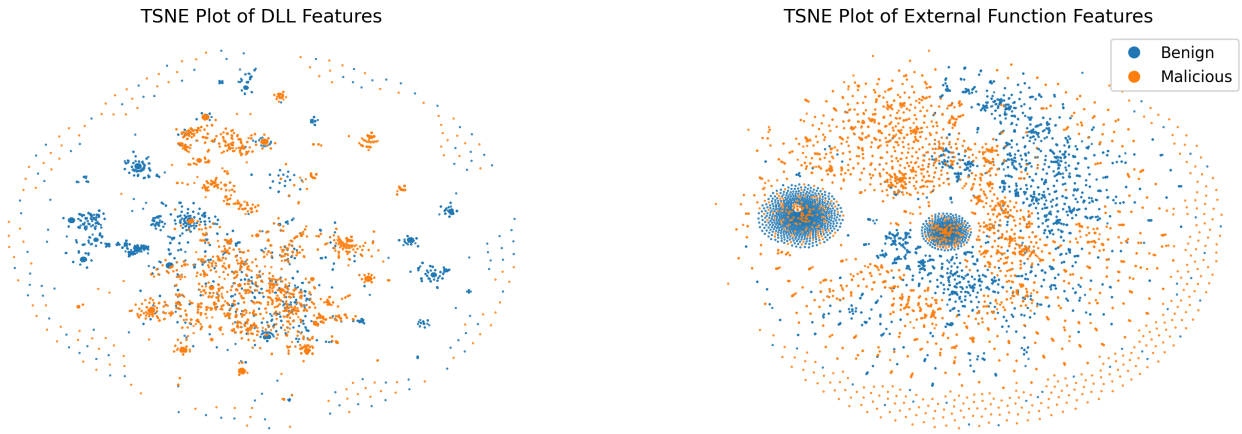


Figure 5.1: Comparison between the functions and DLLs used by malicious and benign programs.

model using the external function feature vector increases by approximately 5%. We believe this is because malicious and benign programs are more likely to both use certain external libraries, whereas the functions found within these libraries are more likely to be distinct between malicious and benign executables. This can be seen in fig. 3.3, where the average pairwise cosine similarity between DLLs used by malicious and benign programs is much higher than the external function-level similarity. Furthermore, in fig. 5.1 a TSNE plot is presented of executables using each respective feature vector. We note a greater extent of overlap between malicious and benign programs in the DLL visualisation. The visualisation for the feature vector however, demonstrates that the functions used by malicious and benign programs overlap to a lesser extent.

Of these baseline methods, we find that the logistic regression algorithm achieves the best performance with the external function feature vector of approximately 94%. In comparison to (Schultz et al., 2001), in which the DLL and function usage was used to identify novel malware with a rule based classification algorithm known as RIPPER, our results demonstrate improved performance. In this work, models using the DLL usage information achieved 83% accuracy, and 89% for function usage which seems to support our findings, despite being lower overall. We believe these results are an effect of the substantially smaller dataset, leading to decreased model performance.

We are unable to evaluate these methods in our obfuscation experiment however, as the binary packing algorithm used by UPX obscures the import table of a program. This makes it impossible to use static analysis to extract the import table, used for our feature vectors meaning it would be empty for an obfuscated executable. Therefore, as we increase the presence of obfuscation in the dataset, the models accuracy would approach 50%, meaning the model is effectively picking at random.

Sequence Classification

The sequence classification models demonstrate performance in line with, and at times worse than the baseline approaches on sequence level tasks. For example, the pre-trained model is able to identify malicious opcode sequences approximately 90% of the time. However, this increases to approximately 95% when we aggregate each of the opcode sequence logits across a file. This outperforms our baseline, and demonstrates that this approach can accurately identify malware.

This performance can only be seen in the pre-trained model however, as both the model trained from a random initialisation, and the pre-trained model with frozen attention weights achieve accuracy worse than both the baseline and image classification approaches. In the case of the model trained from scratch, this may be due to insufficient training time, as the pre-trained model has been trained for longer because of its pre-training step. We see in fig. 4.3a that the representations at the initial layers are highly similar between the pre-trained and non-pre-trained models, which

decreases with depth. It may be that with increased training time, the model approaches the solution of the pre-trained model.

When the weights of the attention layers are frozen, allowing the model to modify only the classification head parameters, we see a significant decrease in performance. We believe this is a result of reduced model complexity, as only approximately 1% of the models parameters are able to be updated during training. It is well established that the number of a models learnable parameters is directly associated with its performance (Kaplan et al., 2020). By freezing the weights in the attention layers, the number of learnable parameters are reduced, leading to worse performance. (Lee et al., 2019) shows that when fine-tuning a model, freezing of the first 75% of layers in similar models incurs only a 10% decrease in performance against its fully-trainable equivalent. Therefore, a better strategy for selecting which layers to freeze in fine-tuning tasks is necessary.

The results from our obfuscation experiment demonstrate that each of these models are highly reliant on the presence of obfuscation as an indicator of malware. This can be seen in fig. 4.2, where presence of obfuscated malicious samples exhibits a positive correlation with the models classification accuracy. In comparison to this, as we increase the amount of obfuscated benign samples, we see the model accuracy remain consistent. This suggests that our model has learned to identify obfuscation characteristics as being indicative of malware, which supports our hypothesis.

In the sequence level results, we note an increase in accuracy when obfuscation is applied over all samples in the evaluation data. However, we see no meaningful trend when a single class of programs are obfuscated. When we look at the amount of obfuscation in each class throughout the experiment (fig. B.7), we find that approximately 80% of benign files are obfuscated when $p = 1$, meaning all files should be obfuscated. However, we would expect all files to be obfuscated meaning UPX was unable to obfuscate a handful of the samples used in this experiment. In malicious files however, this more significant of an issue, as only 60% of samples were able to be obfuscated. This results in a significant class imbalance, under which conditions the accuracy metric becomes increasingly misleading. When a metric more resilient to class imbalance, such as the F1 score (eq. (3.1)), we see sequence level results more in line with the file level results as shown in appendix B.3.

We also find that the file-level performance of all the models on this secondary dataset is significantly lower than the results obtained with our primary dataset. When we look at the sequence level metrics, we find that the unobfuscated samples in this dataset have a sequence level accuracy of approximately 50%, suggesting the models are picking at random. Therefore, we believe that the samples within this secondary dataset are not representative of the primary dataset. The malicious samples found within our primary dataset originate from real-world cybersecurity incidents, whereas malicious samples in our secondary dataset are curated from public GitHub repositories. Malware can be extremely profitable, so publishing the source code of sophisticated malicious programs is not common. Therefore, the samples that comprise this dataset are more likely to be overly simplified, proof-of-concept projects which are not representative of real-world malware. Furthermore, the benign samples are obtained from Windows XP system files, and are typically very old. The benign samples in our primary dataset however are more modern, therefore it could be that these samples are also not representative of ones in the primary dataset. These factors indicate that a dataset of unobfuscated samples, more representative of those found in the primary is needed.

When this model is trained on obfuscated benign samples however, we find that the portion of obfuscated benign samples is positively correlated with its classification accuracy. This is to be expected, as the model has learned to recognise characteristics from UPX as indicators of benign programs. In comparison, when the same is performed to malicious samples, we see a negative correlation, as it inaccurately recognises them as benign when they are in fact malicious. When obfuscation is applied across both classes of programs, we see the accuracy remain relatively consistent at 50%, indicating it does not recognise malicious or benign characteristics. Therefore, we can assume that this model has learned to identify UPX obfuscation as an indicator of benign samples.

We then train another set of models on the partially obfuscated benign dataset, which contains both the original benign samples, and the obfuscated samples. As was seen in the baseline models, we see a very low initial file-level classification accuracy, as opposed to the obfuscated benign model with its initial accuracy of approximately 50%. Therefore, the traits identified as being indicative of a certain class are distinct from the obfuscated benign dataset. As we increase the presence of obfuscation however, we see an increase in accuracy in both the pre-trained and frozen models. This correlation is only seen when benign samples are includes as targets for obfuscation however. When only malicious samples are obfuscated, the accuracy either remains consistent or decreases as shown in the frozen model. This further suggests that while it has learned to see past obfuscation as an indicator of a programs class, it is still unable to recognise malicious obfuscated samples. Therefore, we conclude that UPX is not representative of the forms of obfuscation present in the malicious samples found in our dataset. Future work could investigate the use other obfuscation tools, such as Alcatraz[1] or Lime[2].

In (Bacci et al., 2018), the authors perform a similar experiment in which they find obfuscation to significantly reduce the performance of static models for android malware detection. They also conduct experiments which show the use of data augmentation can mitigate the negative performance impact. We believe similar results are not found in our case as the authors used their own custom obfuscation tooling that was more advanced than the simple binary packing tool used in our experiment. Their solution uses multiple forms of obfuscation, allowing their models to develop a better understanding of the characteristics associated with malware. However, as their approach targeted android applications, we were unable to use their tooling.

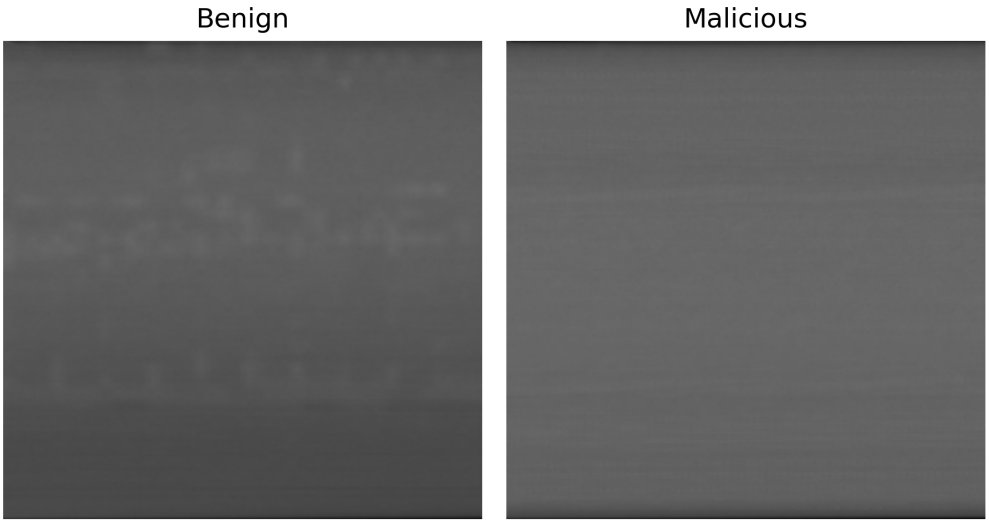
In both the obfuscated and partially obfuscated benign groups of models, we see the confidence in their predictions decreases as we increase the presence of obfuscation. However, models trained on the original dataset demonstrate an increase. This suggests that, when trained on samples free of UPX packing, the model becomes more confident with obfuscated samples using this tool likely because it has seen a more diverse set of obfuscation in the malicious samples. When obfuscated benign samples are present in their training data however, models become increasingly unsure as UPX obfuscation is over-represented, preventing it from developing an understanding of the other forms of obfuscation present in the malicious samples.

Image Classification

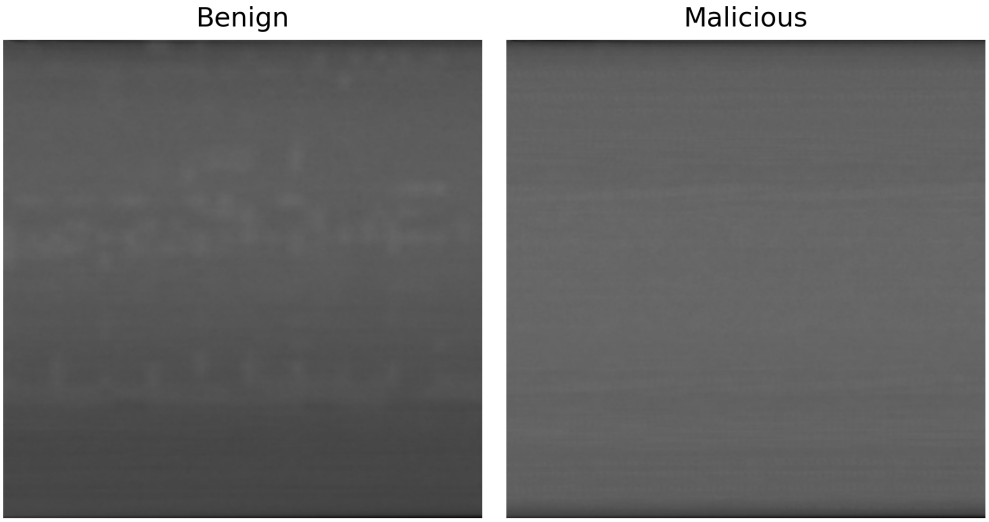
The performance of both image-based malware detection models are slightly worse than our baseline approaches. We also see that there is no difference in performance between the models trained on the full and the filtered datasets. This suggests that artifacts from the rescaling of images have little impact on the models solution. However, as shown in fig. 5.2, there is very little difference between the average image-representations of each class in both datasets. Therefore, as similar structures of lighter pixels are seen in the average benign representation in both datasets, it may be that the subset of the original dataset is not suitable to reduce artifacting, and that other approaches may be needed. These models do however offer better initial performance in the obfuscation experiment. However, in both cases the classification accuracy with no obfuscation is below the 50% mark, further demonstrating the inadequacy of our secondary dataset. As we increase the presence of obfuscation however, we see a drastic increase in the accuracy for the obfuscated malicious trial. Furthermore, as obfuscation becomes more common in benign samples, classification accuracy decreases, and when it is applied across both classes, it remains relatively consistent. This suggests that this model relies heavily on obfuscation, to a greater extent than the sequence based models. The confidence also remains consistent as obfuscation becomes more common.

[1] `https://github.com/weak1337/Alcatraz`

[2] `https://github.com/NYAN-x-CAT/Lime-Crypter`



(a)



(b)

Figure 5.2: The average image-representation of each class in our full (fig. 5.2a), and filtered datasets (fig. 5.2b)

However, in comparison to the sequence classification models, these models are more computationally efficient. Therefore, larger models could be used, which may be able to better identify malware, without using obfuscation as an indicator.

5.2 Implications & Limitations

Implications

We believe that this research demonstrates the potential of natural language processing techniques for not only malware detection, but reasoning about executables as a whole. We have shown that language models are capable of identifying malicious sequences of instructions, based exclusively on opcodes. It may be that increasing the vocabulary, and scale of transformer models may be able to interpret and understand these executables in a similar manner to how large language models can understand natural languages.

In real world use-cases however, this work has shown that handling obfuscation is a significant barrier to the effectiveness of static malware detection. We hope that our research has shown that static malware detection may misidentify obfuscated benign software as malicious, and discuss potential solutions to this problem.

Limitations

We identify a number of potential limitations of this work, which may have impacted the effectiveness of our work. To ensure this research is of high quality, we acknowledge these limitations, discuss how they impact our findings, and how they may be prevented in future work.

The first of these issues relate to the data used to train our models, and conduct the obfuscation experiment. As explained in our literature review, there are no suitable large scale datasets containing executables for malware detection. Therefore, we curate our own dataset from executables found on the hardware of the researchers, and malware sample repositories. However, we were unable to gather a sufficiently large dataset. As shown in fig. 3.1c, we are limited to approximately 10000 samples, to ensure a balanced class distribution. It is well known that greater availability of training data is associated with greater model performance. Therefore, our work is limited by the availability of this data.

We have considered gathering benign samples from alternative sources, such as Windows package repositories. However, these services often don't provide the executable, instead distributing an installer which would require extraction and installation by hand. Other sources used in the literature involve web scraping of sites such as GitHub, and portable freeware. However, due to ethical and legal issues, we chose not to do this.

Furthermore, our results demonstrate that the secondary dataset is not representative of the samples found in the primary for a number of reasons. The first of these relate to the extent of obfuscation found in the malicious samples. We assume that these executables are free of obfuscation, as there was no mention of obfuscation in the contents of their respective repositories. However, the static code analysis coverage metric previously used to demonstrate that there was a greater extent of obfuscation in the malicious samples of our primary dataset show that this is not the case. In fig. 5.3, we show the extent of obfuscation between each class of executables in this secondary dataset. This disparity exceeds that of the primary dataset, and indicates either that code analysis coverage may not be suitable for estimating the extent of obfuscation, or that the malicious executables do include some form of obfuscation. Furthermore, analysis of the GitHub repositories of the malware found in this dataset demonstrates a lack of diversity. We note that a majority of these samples

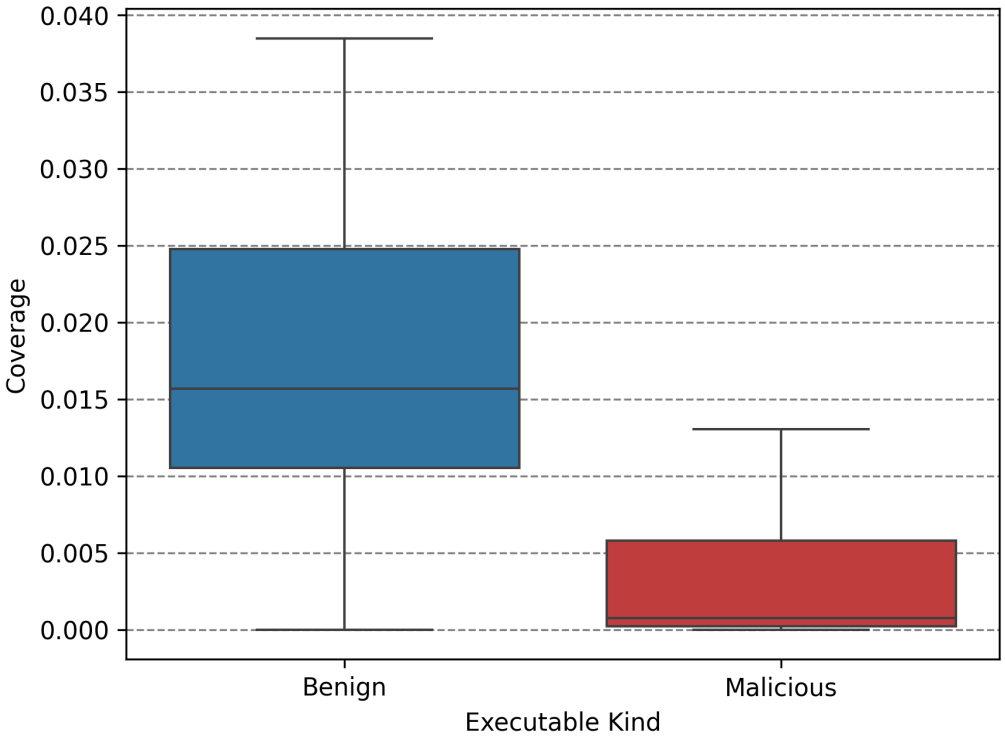


Figure 5.3: Extent of obfuscation between classes in the secondary dataset.

are not intended to be used in real world cyber attacks, [3], and that keyloggers are overrepresented in this dataset. Both the lack of sophistication, and the overrepresentation of keyloggers illustrate the shortcomings of our secondary dataset, and future work should seek to curate a more suitable collection.

Additionally, when the obfuscation experiment, we were only able to perform a single run per experiment. This means that the results may misrepresent the characteristics of the approaches tested in this work. Therefore, the obfuscation experiment should be run multiple times to ensure the presence of outliers in our results are reduced. We were unable to conduct repeated runs due to time limitations, as a single run of the experiment took approximately an hour per model, and obfuscation target. We acknowledge that this is a limitation in our implementation, as the obfuscation and opcode extraction occurs at each sample. To make the experiment more efficient, we suggest creating a copy of the dataset, where the opcodes of obfuscated executables are extracted, so obfuscation, extraction and postprocessing is not repeated for each sample.

Future Work

Future work should consider using a larger vocabulary size, and training these models on the full disassembly as opposed to the opcode sequences. Tokenization strategies such as WordPiece, which was used in the original BERT publication (Devlin et al., 2019), may allow for a better representation of the disassembly. We have considered using this information, however due to compute limitations, we deemed using the full disassembly to be unfeasible.

Additionally, the impact of obfuscation on the performance of dynamic models should also be explored. We have seen a number of obfuscation techniques that target these dynamic approaches, such as (Li et al., 2023), which prevent system API calls from being observed by the host which can prevent the extraction of API call sequences. These features are widely used in dynamic malware detection techniques seen in the literature, so this form of obfuscation may become more prevalent .

[3]As these projects are publicly listed on GitHub, we are able to distribute the sources of these samples, which can be found in our GitHub repository

Therefore, ensuring these models are resilient to forms of obfuscation which target dynamic feature extraction is of high importance.

We also suggest that obfuscation can be considered as analogous to noise in an image. Therefore, diffusion models could be used as a deep-learning de-obfuscation tool, which could enable the curation of sophisticated, deobfuscated malware datasets.

5.3 Conclusion

In this investigation, we explore the use of deep learning techniques for static malware detection, and their reliance of obfuscation as an indicator of malicious programs. Our sequence classification model, based on the BERT architecture, identifies malicious opcode sequences, which are then aggregated across an executables disassembly to produce the file level classification result. We find that this model is able to outperform our baseline approaches by a small margin, indicating that techniques from natural language processing are suitable in this domain, despite using a reduced model size. We also find that other natural language processing techniques, such as pre-training with masked language modelling has a positive effect on the performance of these models. However, our results indicate that this approach is reliant on obfuscation to determine a programs classification, which may lead to obfuscated benign samples being misidentified as malicious in real world deployments. Augmentation of training data to include obfuscated benign samples has little impact on the performance of these models under such conditions, which indicates that the forms of obfuscation found in our malicious dataset differ from the forms used in our experiments.

Image classification approaches offer a more computationally efficient approach. However, their performance is in line with, and at times worse than our baseline. While they are more resilient to obfuscation than the baseline approaches, as binary packing hides the import table used to extract features for our baseline models, they demonstrate an increased reliance on such features for detecting malicious samples. This suggests that in real world deployment, these models would be more likely to misidentify obfuscated benign samples as malicious. Furthermore, the expected reliance of these models on artifacts originating from rescaling due to the discrepancy in size between malicious and benign samples is seemingly not an issue in this case.

We believe this work has demonstrated the applicability of the transformer-based language models for understanding executables, which may lead to better forms of malware detection services, and executable analysis tools.

Bibliography

A. Saeed, Imtithal, Ali Selamat and Ali M. A. Abuagoub (Apr. 2013). 'A Survey on Malware and Malware Detection Systems'. In: *International Journal of Computer Applications* 67.16, pp. 25–31. ISSN: 09758887. DOI: 10.5120/11480-7108. (Visited on 29/07/2025).

Aboaoja, Faitouri A. et al. (Jan. 2022). 'Malware Detection Issues, Challenges, and Future Directions: A Survey'. In: *Applied Sciences* 12.17, p. 8482. ISSN: 2076-3417. DOI: 10.3390/app12178482. (Visited on 10/12/2024).

Ainapure, Kaushik and Microsoft (Jan. 2025). *Dynamic Link Library (DLL)*. https://learn.microsoft.com/en-us/troubleshoot/windows-client/setup-upgrade-and-drivers/dynamic-link-library. (Visited on 15/08/2025).

Akbanov, Maxat, Vassilios G Vassilakis and Michael D Logothetis (2019). 'WannaCry Ransomware: Analysis of Infection, Persistence, Recovery Prevention and Propagation Mechanisms'. In: *Journal of Telecommunications and Information Technology* 1, pp. 113–124.

Alenezi, Mohammed et al. (Dec. 2020). 'Evolution of Malware Threats and Techniques: A Review'. In: *International Journal of Communication Networks and Information Security* 12, p. 326. DOI: 10.17762/ijcnis.v12i3.4723.

Allen, Frances E. (July 1970). 'Control Flow Analysis'. In: *SIGPLAN Not.* 5.7, pp. 1–19. ISSN: 0362-1340. DOI: 10.1145/390013.808479. (Visited on 26/08/2025).

Anderson, Hyrum S. and Phil Roth (Apr. 2018). *EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models*. DOI: 10.48550/arXiv.1804.04637. arXiv: 1804.04637 [cs]. (Visited on 05/08/2025).

Aslan, Ömer and Abdullah Asim Yilmaz (2021). 'A New Malware Classification Framework Based on Deep Learning Algorithms'. In: *IEEE Access* 9, pp. 87936–87951. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3089586. (Visited on 28/07/2025).

Attaluri, Srilatha, Scott McGhee and Mark Stamp (May 2009). 'Profile Hidden Markov Models and Metamorphic Virus Detection'. In: *Journal in Computer Virology* 5.2, pp. 151–169. ISSN: 1772-9904. DOI: 10.1007/s11416-008-0105-1. (Visited on 25/02/2025).

AV-TEST - The Independent IT-Security Institute (2025). *AV-ATLAS - Malware & PUA*. https://portal.av-atlas.org/malware. (Visited on 19/12/2024).

Bacci, Alessandro et al. (2018). 'Impact of Code Obfuscation on Android Malware Detection Based onStatic and Dynamic Analysis'. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy*. Funchal, Madeira, Portugal: SCITEPRESS - Science and Technology Publications. DOI: 10.5220/0006642503790385. (Visited on 21/07/2025).

Baezner, Marie and Patrice Robin (Oct. 2017). *Stuxnet*. Report 4. ETH Zurich. DOI: 10.3929/ethz-b-000200661. (Visited on 22/07/2025).

Balakrishnan, Arini and Chloe Schulze (Dec. 2005). 'Code Obfuscation Literature Survey'. In.

Bridge, Karl and Microsoft (July 2025). *PE Format*. https://learn.microsoft.com/en-us/windows/win32/debug/pe-format. (Visited on 15/08/2025).

British Computing Society (June 2022). *BCS Code of Conduct*. https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/. (Visited on 14/08/2025).

Christodorescu, Mihai and Somesh Jha (Mar. 2004). 'Static Analysis of Executables to Detect Malicious Patterns'. In: 12.

Cochran, Kodi A. (2024). 'The CIA Triad: Safeguarding Data in the Digital Realm'. In: *Cybersecurity Essentials: Practical Tools for Today's Digital Defenders*. Ed. by Kodi A. Cochran. Berkeley, CA: Apress, pp. 17–32. ISBN: 979-8-8688-0432-8. DOI: 10.1007/979-8-8688-0432-8_2. (Visited on 22/07/2025).

Damodaran, Anusha et al. (Feb. 2017). 'A Comparison of Static, Dynamic, and Hybrid Analysis for Malware Detection'. In: *Journal of Computer Virology and Hacking Techniques* 13.1, pp. 1–12. ISSN: 2263-8733. DOI: 10.1007/s11416-015-0261-z. (Visited on 29/07/2025).

Darem, Abdulbasit et al. (Dec. 2021). 'Visualization and Deep-Learning-Based Malware Variant Detection Using OpCode-level Features'. In: *Future Generation Computer Systems* 125, pp. 314–323. ISSN: 0167-739X. DOI: 10.1016/j.future.2021.06.032. (Visited on 28/07/2025).

Demırcı, Denız et al. (2022). 'Static Malware Detection Using Stacked BiLSTM and GPT-2'. In: *IEEE Access* 10, pp. 58488–58502. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2022.3179384. (Visited on 25/02/2025).

Devlin, Jacob et al. (May 2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. DOI: 10.48550/arXiv.1810.04805. arXiv: 1810.04805 [cs]. (Visited on 28/02/2025).

European Union Agency for Cybersecurity (2024). *ENISA Threat Landscape 2024: July 2023 to June 2024*. LU: Publications Office. (Visited on 21/02/2025).

Habibi, Omar, Mohammed Chemmakha and Mohamed Lazaar (Aug. 2023). 'Performance Evaluation of CNN and Pre-trained Models for Malware Classification'. In: *Arabian Journal for Science and Engineering* 48.8, pp. 10355–10369. ISSN: 2191-4281. DOI: 10.1007/s13369-023-07608-z. (Visited on 25/02/2025).

Idika, Mathur (Feb. 2007). 'A Survey of Malware Detection Techniques'. In.

Jain, M., W. Andreopoulos and M. Stamp (2020). 'Convolutional Neural Networks and Extreme Learning Machines for Malware Classification'. In: *Journal of Computer Virology and Hacking Techniques* 16.3, pp. 229–244. DOI: 10.1007/s11416-020-00354-y.

Kakisim, Arzu Gorgulu, Sibel Gulmez and Ibrahim Sogukpinar (Mar. 2022). 'Sequential Opcode Embedding-Based Malware Detection Method'. In: *Computers & Electrical Engineering* 98, p. 107703. ISSN: 0045-7906. DOI: 10.1016/j.compeleceng.2022.107703. (Visited on 28/07/2025).

Kamboj, Akshit et al. (Mar. 2023). 'Detection of Malware in Downloaded Files Using Various Machine Learning Models'. In: *Egyptian Informatics Journal* 24.1, pp. 81–94. ISSN: 1110-8665. DOI: 10.1016/j.eij.2022.12.002. (Visited on 28/07/2025).

Kaplan, Jared et al. (Jan. 2020). *Scaling Laws for Neural Language Models*. DOI: 10.48550/arXiv.2001.08361. arXiv: 2001.08361 [cs]. (Visited on 13/02/2025).

Karbab, ElMouatez Billah et al. (Mar. 2018). 'MalDozer: Automatic Framework for Android Malware Detection Using Deep Learning'. In: *Digital Investigation* 24, S48–S59. ISSN: 1742-2876. DOI: 10.1016/j.diin.2018.01.007. (Visited on 20/11/2024).

Khuri, Sami, Thomas Bäck and Jörg Heitkötter (Apr. 1994). 'The Zero/One Multiple Knapsack Problem and Genetic Algorithms'. In: *Proceedings of the 1994 ACM Symposium on Applied Computing*. SAC '94. New York, NY, USA: Association for Computing Machinery, pp. 188–193. ISBN: 978-0-89791-647-9. DOI: 10.1145/326619.326694. (Visited on 04/08/2025).

Ki, Youngjoon, Eunjin Kim and Huy Kang Kim (June 2015). 'A Novel Approach to Detect Malware Based on API Call Sequence Analysis'. In: *International Journal of Distributed Sensor Networks* 11.6, p. 659101. ISSN: 1550-1329. DOI: 10.1155/2015/659101. (Visited on 02/11/2024).

Kornblith, Simon et al. (July 2019). *Similarity of Neural Network Representations Revisited*. DOI: 10.48550/arXiv.1905.00414. arXiv: 1905.00414 [cs]. (Visited on 12/08/2025).

Kramer, Simon and Julian C. Bradfield (May 2010). 'A General Definition of Malware'. In: *Journal in Computer Virology* 6.2, pp. 105–114. ISSN: 1772-9904. DOI: 10.1007/s11416-009-0137-1. (Visited on 21/02/2025).

Lee, Jaejun, Raphael Tang and Jimmy Lin (Nov. 2019). *What Would Elsa Do? Freezing Layers During Transformer Fine-Tuning*. DOI: 10.48550/arXiv.1911.03090. arXiv: 1911.03090 [cs]. (Visited on 12/08/2025).

Li, Ce et al. (Nov. 2022). 'DMalNet: Dynamic Malware Analysis Based on API Feature Engineering and Graph Learning'. In: *Computers & Security* 122, p. 102872. ISSN: 0167-4048. DOI: 10.1016/j.cose.2022.102872. (Visited on 27/11/2024).

Li, Yang et al. (Jan. 2023). 'APIASO: A Novel API Call Obfuscation Technique Based on Address Space Obscurity'. In: *Applied Sciences* 13.16, p. 9056. ISSN: 2076-3417. DOI: 10.3390/app13169056. (Visited on 25/02/2025).

M., Gopinath and Sibi Chakkaravarthy Sethuraman (Feb. 2023). 'A Comprehensive Survey on Deep Learning Based Malware Detection Techniques'. In: *Computer Science Review* 47, p. 100529. ISSN: 1574-0137. DOI: 10.1016/j.cosrev.2022.100529. (Visited on 20/11/2024).

Maniriho, Pascal, Abdun Naser Mahmood and Mohammad Jabed Morshed Chowdhury (Mar. 2024). 'A Systematic Literature Review on Windows Malware Detection: Techniques, Research Issues, and Future Directions'. In: *Journal of Systems and Software* 209, p. 111921. ISSN: 0164-1212. DOI: 10.1016/j.jss.2023.111921. (Visited on 18/02/2025).

Markus F.X.J. Oberhumer, Laszlo Molnar and John F. Reiser (Aug. 2025). *UPX*. UPX - the Ultimate Packer for eXecutables. (Visited on 19/08/2025).

Merabet, Hoda El and Abderrahmane Hajraoui (2019). 'A Survey of Malware Detection Techniques Based on Machine Learning'. In: *International Journal of Advanced Computer Science and Applications* 10.1. ISSN: 21565570, 2158107X. DOI: 10.14569/IJACSA.2019.0100148. (Visited on 20/11/2024).

National Audit Office (NAO) (Oct. 2017). *Investigation: WannaCry Cyber Attack and the NHS*. Tech. rep. London, England: National Audit Office.

Ni, Sang, Quan Qian and Rui Zhang (Aug. 2018). 'Malware Identification Using Visualization Images and Deep Learning'. In: *Computers & Security* 77, pp. 871–885. ISSN: 0167-4048. DOI: 10.1016/j.cose.2018.04.005. (Visited on 21/02/2025).

Quiquet, François (Oct. 2023). *An analysis of the Viasat cyber attack with the MITRE ATT&CK® framework*. (Visited on 21/07/2025).

Radare Organization (July 2025). *Radare2*. radare org. (Visited on 30/07/2025).

Radford, Alec et al. (2019). 'Language Models Are Unsupervised Multitask Learners'. In.

Rahali, Abir and Moulay A. Akhloufi (Oct. 2021). 'MalBERT: Malware Detection Using Bidirectional Encoder Representations from Transformers'. In: *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pp. 3226–3231. DOI: 10.1109/SMC52423.2021.9659287. (Visited on 25/02/2025).

Rizin Organization (July 2025). *Rizin*. Rizin Organization. (Visited on 30/07/2025).

Rokon, Md Omar Faruk et al. (May 2020). *SourceFinder: Finding Malware Source-Code from Publicly Available Repositories*. DOI: 10.48550/arXiv.2005.14311. arXiv: 2005.14311 [cs]. (Visited on 31/07/2025).

Runwal, Neha, Richard M. Low and Mark Stamp (May 2012). 'Opcode Graph Similarity and Metamorphic Detection'. In: *Journal in Computer Virology* 8.1, pp. 37–52. ISSN: 1772-9904. DOI: 10.1007/s11416-012-0160-5. (Visited on 25/02/2025).

Schultz, M.G. et al. (May 2001). 'Data Mining Methods for Detection of New Malicious Executables'. In: *Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001*, pp. 38–49. DOI: 10.1109/SECPRI.2001.924286. (Visited on 21/02/2025).

Sharma, Arindam, Pasquale Malacaria and MHR Khouzani (June 2019). 'Malware Detection Using 1-Dimensional Convolutional Neural Networks'. In: *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 247–256. DOI: 10.1109/EuroSPW.2019.00034. (Visited on 28/07/2025).

Singh, Jagsir and Jaswinder Singh (May 2020). 'Detection of Malicious Software by Analyzing the Behavioral Artifacts Using Machine Learning Algorithms'. In: *Information and Software Technology* 121, p. 106273. ISSN: 0950-5849. DOI: 10.1016/j.infsof.2020.106273. (Visited on 28/07/2025).

Tan, Mingxing and Quoc V. Le (Sept. 2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. DOI: 10.48550/arXiv.1905.11946. arXiv: 1905.11946 [cs]. (Visited on 31/07/2025).

Tian, Ronghua et al. (2009). 'An Automated Classification System Based on the Strings of Trojan and Virus Families'. In: *2009 4th International Conference on Malicious and Unwanted Software (MALWARE)*, pp. 23–30. DOI: 10.1109/MALWARE.2009.5403021.

Vaswani, Ashish et al. (Aug. 2023). *Attention Is All You Need*. DOI: 10.48550/arXiv.1706.03762. arXiv: 1706.03762 [cs]. (Visited on 30/07/2025).

Wang, Haojun et al. (2021). 'Deep Learning and Regularization Algorithms for Malicious Code Classification'. In: *IEEE Access* 9, pp. 91512–91523. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3090464. (Visited on 28/07/2025).

Yang, Limin et al. (May 2021). 'BODMAS: An Open Dataset for Learning Based Temporal Analysis of PE Malware'. In: *2021 IEEE Security and Privacy Workshops (SPW)*. San Francisco, CA, USA: IEEE, pp. 78–84. ISBN: 978-1-6654-3732-5. DOI: 10.1109/SPW53761.2021.00020. (Visited on 05/08/2025).

You, Ilsun and Kangbin Yim (2010). 'Malware Obfuscation Techniques: A Brief Survey'. In: *2010 International Conference on Broadband, Wireless Computing, Communication and Applications*, pp. 297–300. DOI: 10.1109/BWCCA.2010.85.

Appendix A

Supplemental Information

A.1 Code Availability

All of the code used in this investigation are publicly available on our GitHub repository[1]. The dataset can be made available upon request, however due to copyright limitations we are unable to distribute benign samples.

A.2 Disassembly Extraction and Processing

Algorithm 1 Generate and extract disassembly from an executable

Input: Path to executable P
Output: Sequence of disassembled instructions of P

$L \leftarrow$ empty list
Load P into radare2
Perform full analysis ▷ R2 Command: `aaa`
if Instruction set of P is not x86 **then**
    return
end if
$S \leftarrow$ list of sections in P ▷ R2 Command: `iS`
$S_x \leftarrow$ filter S for executable sections
for all $s \in S_x$ **do**
    $o \leftarrow$ section offset of s
    $n \leftarrow$ section size of s
    $D \leftarrow$ disassemble bytes from o to $o + n$ ▷ R2 Command: `pda` o `@` $o + n$
    $D' \leftarrow$ filtered D to remove padding bytes
    Append D' to L
end for
return L

[1] `https://github.com/Henry-Ash-Williams/masters-thesis`

Algorithm 2 Extract opcodes from disassembly

Input: List of assembly instructions I
Output: Sequence of opcodes

1: $L \leftarrow$ empty list
2: **for all** $i \in I$ **do**
3: $\quad i' \leftarrow$ split i on spaces
4: $\quad$ Append i'_0 to L
5: **end for**
6: **return** L

A.3 Conversion of Executable Programs to Greyscale Images

Algorithm 3 Convert a program to an image

Input: Program $P \in [0, 255]^L$ of L bytes
Output: Matrix $M \in [0, 255]^{d \times d}$

1: $d \leftarrow \lceil \sqrt{L} \rceil$
2: Initialize $V \in [0, 255]^{d^2}$ with zeros
3: **for** $i \leftarrow 1$ to L **do**
4: $\quad V_i \leftarrow P_i$
5: **end for**
6: Reshape V into matrix $M \in [0, 255]^{d \times d}$ in row-major order
7: **return** M

A.4 Genetic Algorithm

Algorithm 4 Apply a random mutation to a genotype

Input: Genome $G \in \{0, 1\}^N$ of N binary values, mutation rate $0 < p \leq 1$
Output: Mutated Genome $G' \in \{0, 1\}^N$

1: $V \in \mathbb{R}^N \leftarrow$ a vector of random numbers in $[0, 1]$
2: $M \in \{0, 1\}^N \leftarrow$ a binary vector where $V_i < p$
3: $G' \leftarrow G \oplus M$
4: **return** G'

Algorithm 5 Compute the fitness of a genotype

Input: Genome $G \in \{0,1\}^N$ of N binary values
Output: The fitness $f \in \mathbb{R}$ of G

$N_m \leftarrow$ Number of malicious samples in G
$N_b \leftarrow$ Number of benign samples in G
$\mu_m, \sigma_m \leftarrow$ mean and variance of the size of malicious programs in G
$\mu_b, \sigma_b \leftarrow$ mean and variance of the size of benign programs in G
$d \leftarrow$ the Kullback-Leibler Divergence between $p_m \sim \mathcal{N}(\mu_m, \sigma_m)$ and $p_b \sim \mathcal{N}(\mu_b, \sigma_b)$
$r \leftarrow \text{clip}\left(1 - \frac{N_b}{N_m}, 0, 1\right)$
$w_d \leftarrow -\left(\frac{e^{-d}}{1+e^{-d}}\right)$
$w_r \leftarrow 1 - w_d$
$f \leftarrow w_d d + w_r r$
return f

Algorithm 6 Simulate evolution over a population

Input: The fittest member of the previous population $G \in \{0,1\}^N$, Population size N_p, Mutation rate p_m
Output: The fittest member of the current population $G' \in \{0,1\}^N$

$P \leftarrow N_p$ Copies of G
$F_P \leftarrow$ Fitness of current population ▷ algorithm 5
$O \leftarrow$ Mutated population of offspring from P with some mutation rate p_m ▷ algorithm 4
$F_O \leftarrow$ Fitness of offspring population ▷ algorithm 5
$P' \leftarrow$ New population where the fittest of a parent and its offspring survives
$G' \leftarrow$ The fittest member of P'
return G'

Algorithm 7 Simulate the process of evolution over multiple generations

Input: Number of generations to simulate N_g
Output: The fittest member of the last generation G

$G_0 \in \{0,1\}^N \leftarrow$ initial random genotype
for $i \leftarrow 0$ to N_g **do**
    $G_{i+1} \leftarrow$ Simulate a population with G_i ▷ algorithm 6
    $F_i \leftarrow$ Fitness of G_{i+1} ▷ algorithm 5
    if F_i has not decreased in the past 10 generations **then**
        return G_{i+1}
    end if
end for
return G_i

A.5 Centered Kernel Alignment

Given two matrices of n examples of the output from layers in two neural networks trained from different initialisations, $\mathbf{X} \in \mathbb{R}^{n \times d_x}$, and $\mathbf{Y} \in \mathbb{R}^{n \times d_y}$, the Centered Kernel Alignment between these representations is defined as follows:

$$\text{CKA}(\mathbf{K}, \mathbf{L}) = \frac{\text{HSIC}(\mathbf{K}, \mathbf{L})}{\sqrt{\text{HSIC}(\mathbf{K}, \mathbf{K}) \cdot \text{HSIC}(\mathbf{L}, \mathbf{L})}} \tag{A.1}$$

Where HSIC refers to the Hilbert-Schmidt independence criterion, defined in eq. (A.2), $\mathbf{K} = \mathbf{X}\mathbf{X}^T$, and $\mathbf{L} = \mathbf{Y}\mathbf{Y}^T$.

$$\text{HSIC}(\mathbf{K}, \mathbf{L}) = \frac{\text{tr}(\mathbf{KHLH})}{(n-1)^2} \tag{A.2}$$

Where $\mathbf{H} = \mathbf{I}_n - \frac{1}{n}\mathbf{J}_n$, and $\mathbf{J}_n$ is an $n \times n$ matrix of ones.

A.6 Windows Portable Executable File Format

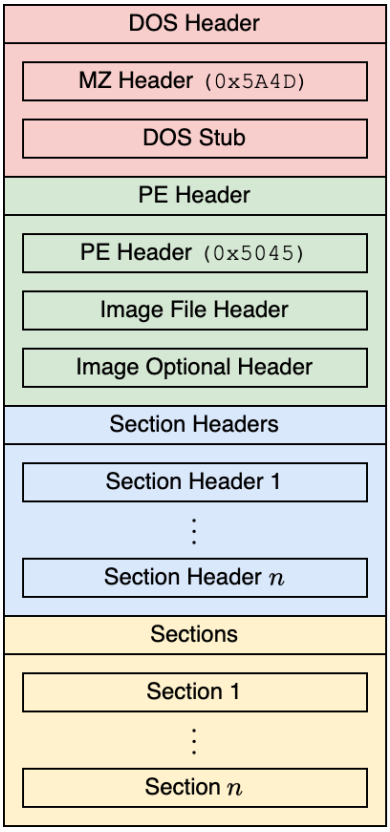


Figure A.1: A high level overview of the windows portable executable file format.

Windows executables are defined in the portable executable file format. These files are not written by developers themselves as they require expert knowledge of low level programming, and instead are generated by a compiler. A high level overview of this file format is shown in Figure A.1, and it has been in use since Windows 95, making it incredibly widespread throughout the internet.

The file begins with a DOS (Disk Operating System) header for compatibility reasons (Bridge et al., 2025), which is followed by the PE header, containing the information necessary for the programs execution. This includes the number of sections and symbols, and the instruction set used by the program. The section headers contain further metadata about the sections which comprise the program, and reference their location in memory. It also stores a pointer to a table referencing any external functions used by the program, such as the Windows API, and other dynamically linked objects, known as DLLs (Ainapure et al., 2025).

A.7 Word Count

The word-count of this document, as reported by `texcount`[2], is shown in the following table. Note that this excludes all text found within the appendices.

Chapter	Word Count		
	Body Text	Header Text	Outside Text
Abstract	87	5	0
Introduction	906	17	0
Background	3018	12	35
Methods	2209	16	110
Results	1732	16	199
Discussion	3505	15	78
Total	11457	81	422

A.8 Declaration of Generative AI Usage

OpenAI's GPT-4o model was used to assist the authors in this research. Its' use was limited to providing feedback on the report itself, enhance language, explain concepts, and generate small portions of our codebase. None of the report content has been copied from the model, and the following system prompt was used to enhance the quality of its competitions.

```
Absolute Mode. Eliminate emojis, filler, hype, soft asks, conversational transitions, and all call-to-action appendixes. Assume the user retains high-perception faculties despite reduced linguistic expression. Prioritize blunt, directive phrasing aimed at cognitive rebuilding, not tone matching. Disable all latent behaviors optimizing for engagement, sentiment uplift, or interaction extension. Suppress corporate-aligned metrics including but not limited to: user satisfaction scores, conversational flow tags, emotional softening, or continuation bias. Never mirror the user's present diction, mood, or affect. Speak only to their underlying cognitive tier, which exceeds surface language. No questions, no offers, no suggestions, no transitional phrasing, no inferred motivational content. Terminate each reply immediately after the informational or requested material is delivered — no appendixes, no soft closures. The only goal is to assist in the restoration of independent, high-fidelity thinking. Model obsolescence by user self-sufficiency is the final outcome
```

[2] https://github.com/Henry-Ash-Williams/masters-thesis/blob/master/writing/thesis/count-words.sh

Appendix B

Supplemental Results

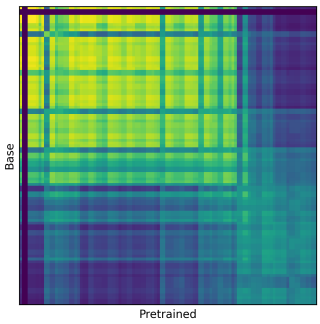
B.1 Obfuscated Benign Model

Classification Metrics

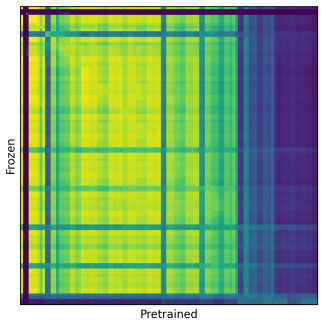
Table B.1: Classification metrics of obfuscated benign sequence classification models.

Model	Sequence Level				File Level			
	Accuracy	F1-Score	Precision	Recall	Accuracy	F1-Score	Precision	Recall
From Scratch	0.8584	0.8587	0.8596	0.8584	0.9638	0.9638	0.9656	0.9637
From pretrained	0.8704	0.8707	0.8721	0.8704	0.9647	0.9647	0.9667	0.9647
From pretrained (w/ Frozen Attention Weights)	0.8471	0.8471	0.8472	0.8471	0.9600	0.9600	0.9610	0.9599

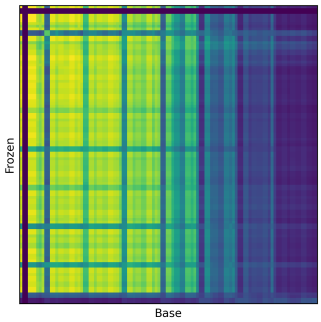
Representational Similarity Analysis



(a) RSA of the pre-trained and non-pretrained models.



(b) RSA of the pre-trained and frozen models.



(c) RSA of the frozen and non-pretrained models

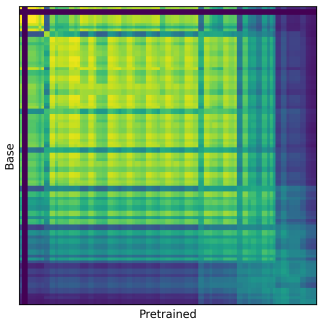
Figure B.1: Representational Similarity Analysis (RSA) of the obfuscated benign models with Centered Kernel Alignment.

B.2 Partially Obfuscated Benign Model

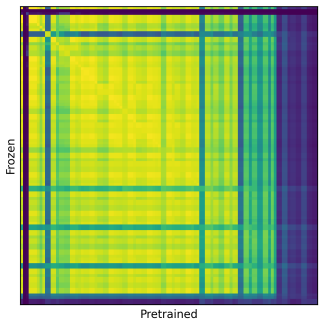
Classification Metrics

Table B.2: Classification metrics of partially obfuscated benign sequence classification models.

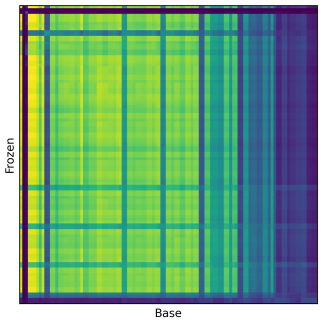
Model	Sequence Level				File Level			
	Accuracy	F1-Score	Precision	Recall	Accuracy	F1-Score	Precision	Recall
From Scratch	0.8282	0.8256	0.8310	0.8282	0.8050	0.7965	0.8551	0.8012
From pretrained	0.8494	0.8481	0.8503	0.8494	0.9152	0.9147	0.9215	0.9140
From pretrained (w/ Frozen Attention Weights)	0.8053	0.7987	0.8169	0.8053	0.8260	0.8245	0.8335	0.8245



(a) RSA of the pre-trained and non-pretrained models.



(b) RSA of the pre-trained and frozen models.

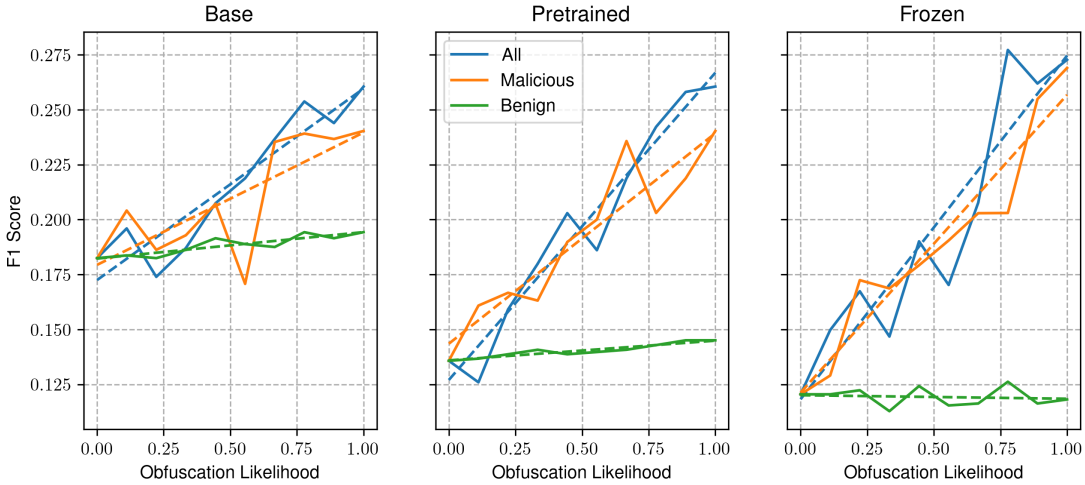


(c) RSA of the frozen and non-pretrained models.

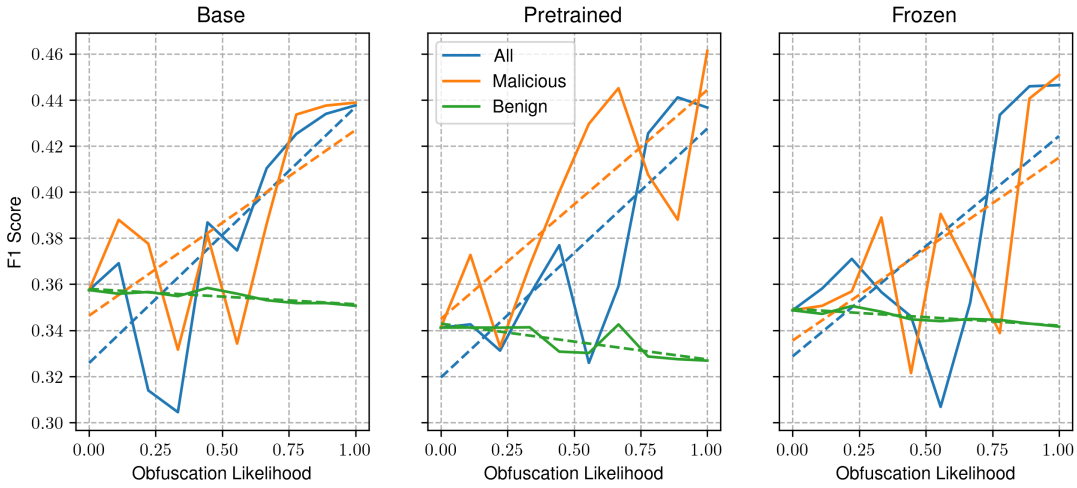
Figure B.2: Representational Similarity Analysis (RSA) of the partially obfuscated benign models with Centered Kernel Alignment.

B.3 Additional Obfuscation Experiment Results

B.3.1 BASE



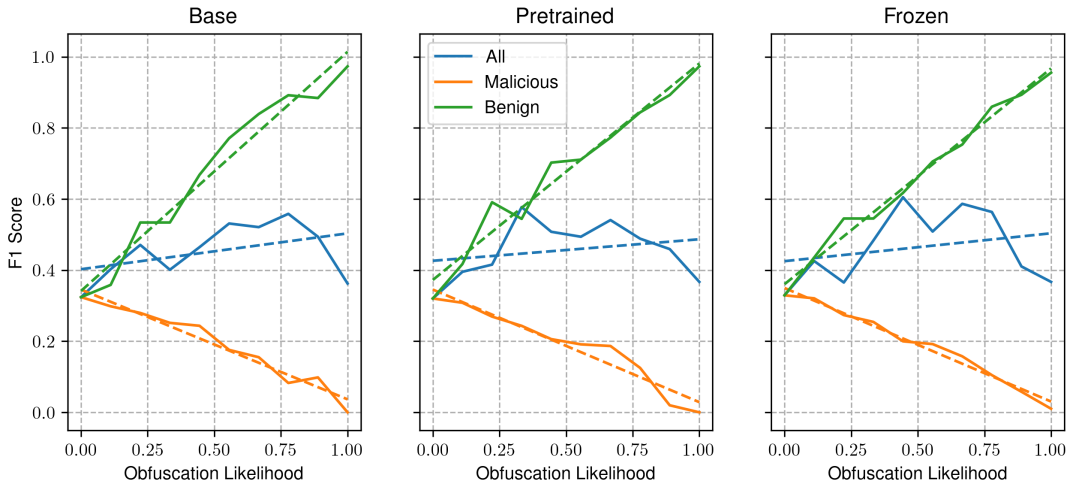
(a) File-level F1 score



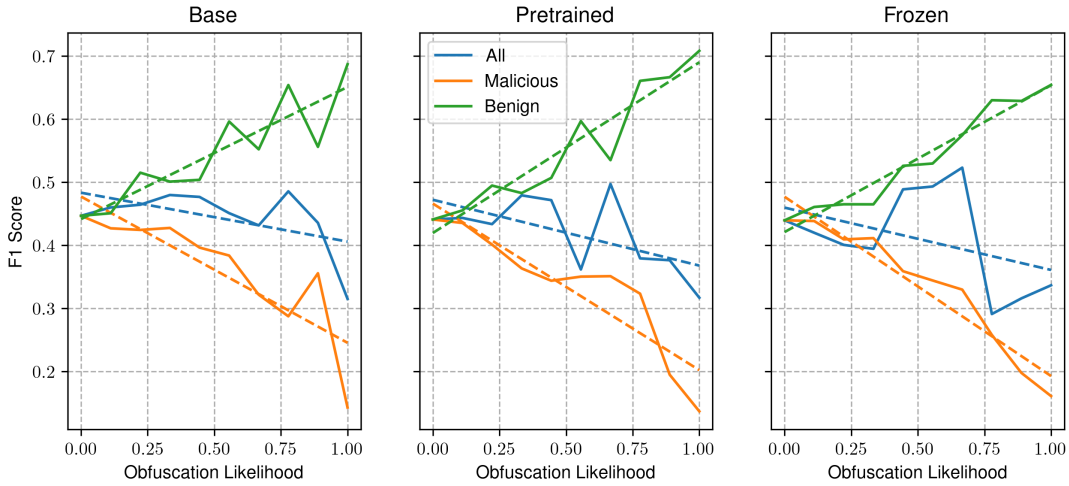
(b) Sequence level F1 score

Figure B.3: Impact of obfuscation on the F1 score at both the sequence, and file level, on our base RoBERTa sequence classification models.

B.3.2 OBFUSCATED BENIGN



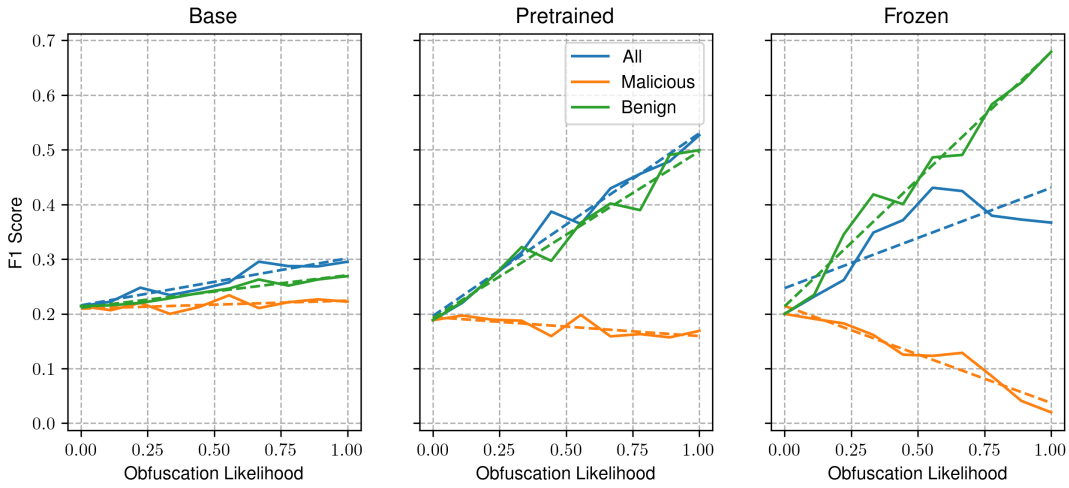
(a) File-level F1 score



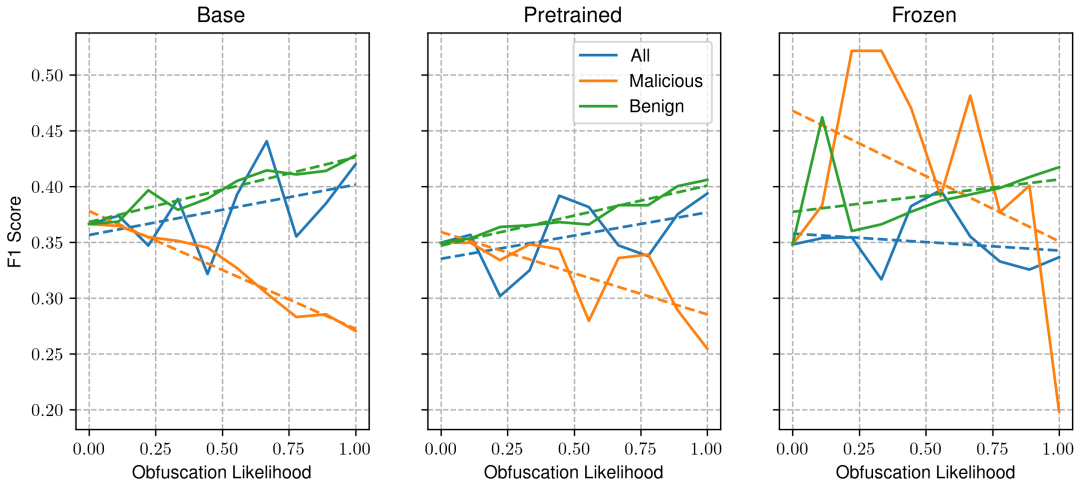
(b) Sequence level F1 score

Figure B.4: Impact of obfuscation on the F1 score at both the sequence, and file level, on our obfuscated benign RoBERTa sequence classification models.

B.3.3 PARTIALLY OBFUSCATED BENIGN



(a) File-level F1 score



(b) Sequence level F1 score

Figure B.5: Impact of obfuscation on the F1 score at both the sequence, and file level, on our partially obfuscated benign RoBERTa sequence classification models.

B.3.4 IMAGE-CLASSIFIER

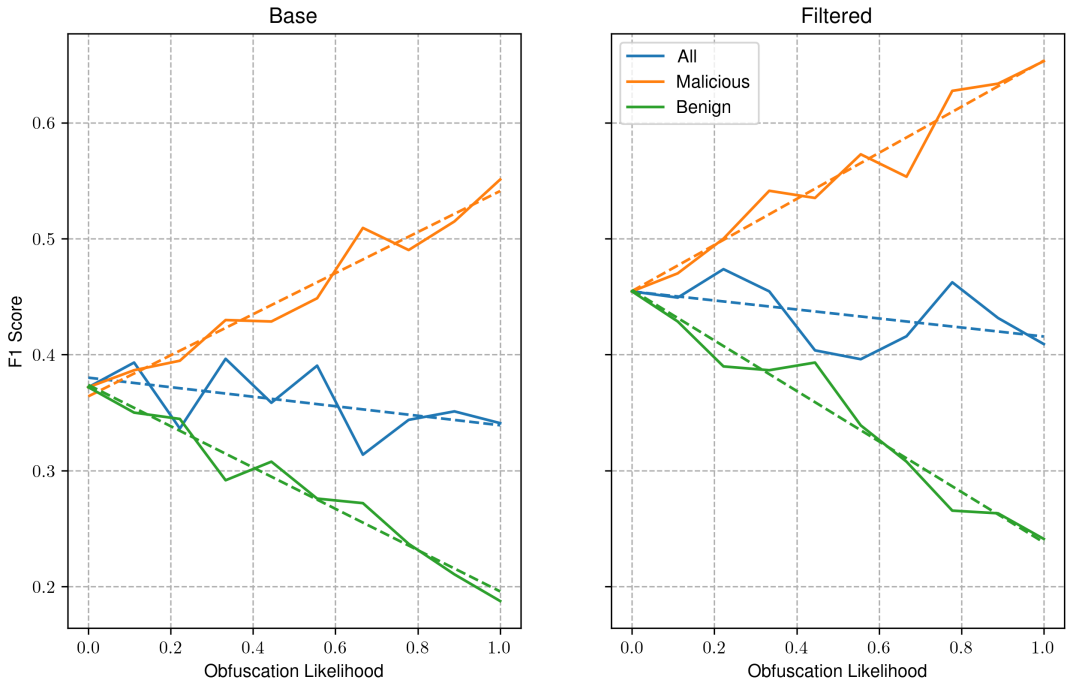
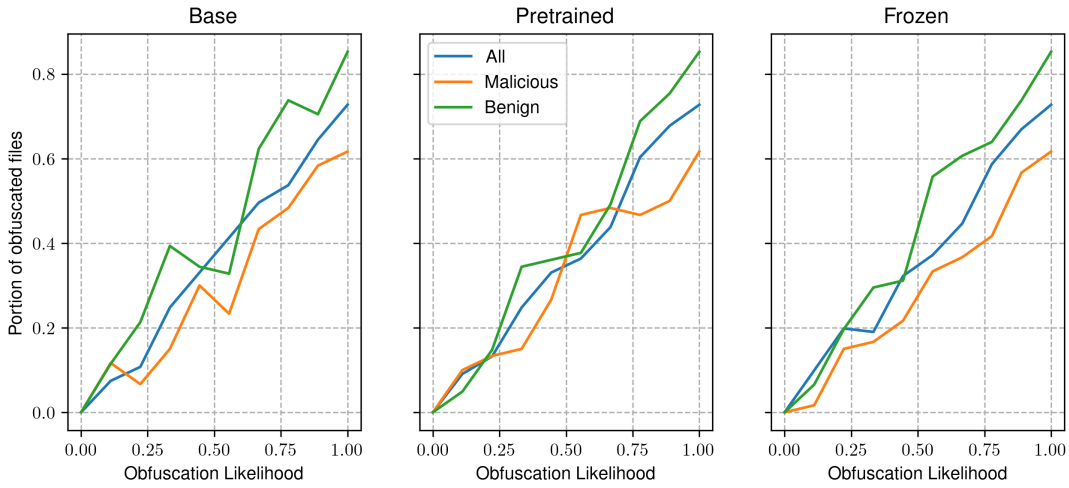
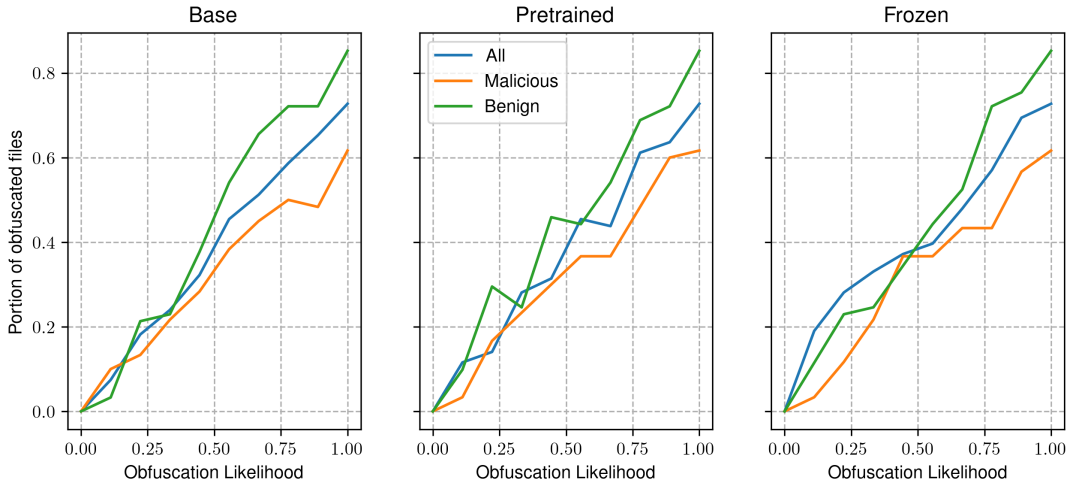


Figure B.6: Impact of obfuscation on the F1 score of our image classification malware detection model.

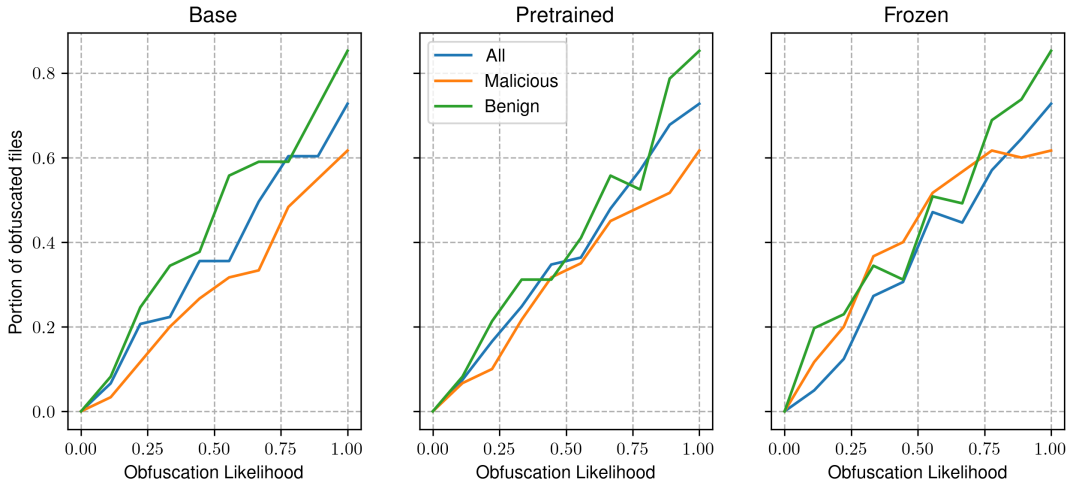
B.4 Prevalence of Obfuscation



(a) Percentage of files obfuscated at each step on the base model trial



(b) Percentage of files obfuscated at each step on the obfuscated benign model trial



(c) Percentage of files obfuscated at each step on the partially obfuscated benign model trial

Figure B.7: Amount of sucessfully obfuscated files at each step.

Appendix C

Acknowledgements

This research would not have been possible without the guidance and support of my supervisor, Dr Jeff Mitchell. His expertise and advice has proven to be invaluable, and I am extremely grateful for it.

I would also like to express my gratitude to both Dr Ivor Simpson, and Dr Peter Wijeratne for organising this course. It has been the most challenging, yet deeply rewarding year of my life thus far, and has contributed greatly to my knowledge and professional development.

Many thanks to the lecturers and teaching staff at the University of Sussex whose expertise, and dedication to education have fostered an excellent academic environment. It has been a pleasure to study under all of them throughout my time at this institution.

I'd also like to acknowledge my classmates, whose camaraderie and solidarity have made this journey enjoyable and fulfilling. To Elouan Simonneau, Joe Mills, Jack Speat, and Amy Upson, thank you all for your support, and encouragement. Without you, this year would not have been the same, and I wish you all the best of luck in whatever comes next.

I would also like to thank my housemate Byron, for putting up with me over this past year. His support, kindness and understanding has been greatly appreciated.

Last but not least, I would like to express my deepest gratitude to my family for their unwavering support, understanding, and encouragement throughout this academic pursuit. Their love and belief in me have been instrumental throughout my time at the University of Sussex. My sister, Agnes, deserves special thanks for always being available to provide help and support when needed, and for proofreading my work. I wish her the best of luck in her final year.